

BAN404 – Statistical Machine Learning and Data Analysis

Exam Compendium | Based on ISLP

Exam: May 19, 2026

Contents

1	Chapter 1 – Introduction to Statistical Learning	5
1.1	What is Statistical Learning?	5
1.2	Notation	5
1.3	Why Do We Estimate f ?	5
1.3.1	Prediction	5
1.3.2	Inference	6
1.4	Parametric vs. Non-Parametric Methods	6
1.4.1	Parametric Methods	6
1.4.2	Non-Parametric Methods	6
1.5	Supervised vs. Unsupervised Learning	7
2	Chapter 2 – Assessing Model Accuracy	9
2.1	Measuring the Quality of Fit	9
2.1.1	The Pattern of Training and Test MSE as Flexibility Increases	9
2.2	The Bias-Variance Trade-off	9
2.3	The Classification Setting	10
2.3.1	The Bayes Classifier	10
2.3.2	K-Nearest Neighbours Classifier	11
2.4	Summary: Key Principles of Model Assessment	11
3	Chapter 3 – Linear Regression	13
3.1	Simple Linear Regression	13
3.1.1	Estimating Coefficients — Ordinary Least Squares (OLS)	13
3.1.2	Assessing the Accuracy of Coefficient Estimates	13
3.1.3	Assessing Model Accuracy: RSE, R^2 , and the F -statistic	14
3.1.4	Confidence Intervals vs. Prediction Intervals	15
3.2	Multiple Linear Regression	15
3.2.1	Interpretation of Coefficients in Multiple Regression	16
3.2.2	Key Questions in Multiple Regression	16
3.3	Extensions of the Linear Model	16
3.3.1	Interaction Terms	16
3.3.2	Polynomial Regression	16
3.3.3	Qualitative (Categorical) Predictors	16
3.4	Potential Problems in Linear Regression	17
3.4.1	1. Non-linearity of the Relationship	17
3.4.2	2. Heteroscedasticity (Non-constant Error Variance)	17
3.4.3	3. Correlated Errors (Autocorrelation)	17
3.4.4	4. Outliers	17
3.4.5	5. High-Leverage Points	17
3.4.6	6. Multicollinearity	18
3.5	KNN Regression vs. Linear Regression	18
4	Chapter 4 – Classification	20
4.1	Why Not Linear Regression for Classification?	20
4.2	Logistic Regression	20
4.2.1	The Logistic Model	20
4.2.2	Estimating Coefficients — Maximum Likelihood	21

4.2.3	Multiple Logistic Regression	21
4.2.4	Multinomial Logistic Regression ($K > 2$ Classes)	21
4.3	Evaluating Classifiers	21
4.3.1	The Confusion Matrix	21
4.3.2	The ROC Curve and AUC	22
4.3.3	Cost-Optimal Classification Threshold	22
4.4	Generative Models for Classification	23
4.5	Linear Discriminant Analysis (LDA)	23
4.5.1	LDA for $p = 1$ (Single Predictor)	23
4.5.2	LDA for $p > 1$ (Multiple Predictors)	23
4.6	Quadratic Discriminant Analysis (QDA)	24
4.6.1	LDA vs. QDA: When to Use Which	24
4.7	Naive Bayes	24
4.8	K-Nearest Neighbours Classifier (Revisited)	25
4.9	Comparison of Classification Methods	25
5	Chapter 5 – Resampling Methods	27
5.1	Cross-Validation	27
5.1.1	The Validation Set Approach	27
5.1.2	Leave-One-Out Cross-Validation (LOOCV)	27
5.1.3	k -Fold Cross-Validation	28
5.1.4	Bias-Variance Trade-off for k -Fold CV	28
5.1.5	CV for Model Selection	28
5.1.6	Cross-Validation for Classification	29
5.2	The Bootstrap	29
5.2.1	Core Idea and Algorithm	29
5.2.2	What Observations End Up in a Bootstrap Sample?	29
5.2.3	The Portfolio Example: When Bootstrap is Essential	30
5.3	Cross-Validation vs. Bootstrap: A Critical Distinction	30
6	Chapter 6 – Linear Model Selection and Regularization	32
6.1	Subset Selection	32
6.1.1	Best Subset Selection	32
6.1.2	Forward Stepwise Selection	32
6.1.3	Backward Stepwise Selection	32
6.2	Choosing the Optimal Model Size	32
6.2.1	Information Criteria	33
6.2.2	Cross-Validation for Model Selection	33
6.3	Shrinkage Methods: Ridge and Lasso	33
6.3.1	Ridge Regression (L2 Regularisation)	33
6.3.2	The Lasso (L1 Regularisation)	34
6.3.3	Ridge vs. Lasso: A Detailed Comparison	34
6.3.4	Selecting λ by Cross-Validation	35
6.4	Dimension Reduction: Principal Components Regression (PCR)	35
7	Chapter 7 – Moving Beyond Linearity	37
7.1	Polynomial Regression	37
7.2	Step Functions	37
7.3	Basis Functions: A Unifying Framework	37
7.4	Regression Splines	38
7.4.1	Piecewise Polynomials	38
7.4.2	Cubic Splines	38
7.4.3	Natural Splines	38
7.4.4	Choosing the Number and Location of Knots	39
7.5	Smoothing Splines	39
7.6	Local Regression	40
7.7	Generalised Additive Models (GAMs)	40
7.7.1	Fitting GAMs: Backfitting	40
7.7.2	GAMs for Classification (Logistic GAMs)	41
7.7.3	Advantages and Disadvantages of GAMs	41

8	Chapter 8 – Tree-Based Methods	43
8.1	Regression Trees	43
8.1.1	Building a Regression Tree: Recursive Binary Splitting	43
8.1.2	Tree Pruning: Cost-Complexity Pruning	43
8.2	Classification Trees	44
8.3	Advantages and Disadvantages of Trees	45
8.4	Bagging (Bootstrap Aggregating)	45
8.4.1	Out-of-Bag (OOB) Error	45
8.4.2	Variable Importance from Bagged Trees	46
8.5	Random Forests	46
8.6	Gradient Boosting	47
8.6.1	The Boosting Idea	47
8.6.2	The Three Hyperparameters of Boosting	47
8.6.3	Why Does Boosting Work? The Bias Perspective	48
8.7	Summary of Ensemble Methods	48
9	Chapter 9 – Support Vector Machines	50
9.1	What is a Hyperplane?	50
9.2	The Maximal Margin Classifier (Hard Margin SVM)	50
9.2.1	Support Vectors	50
9.2.2	Limitations of the Maximal Margin Classifier	51
9.3	The Support Vector Classifier (Soft Margin SVM)	51
9.3.1	Interpreting the Slack Variables	51
9.3.2	The Role of C (Budget Parameter)	51
9.4	The Support Vector Machine (Kernel Trick)	51
9.4.1	The Dual Formulation and Inner Products	52
9.4.2	Common Kernel Functions	52
9.4.3	Choosing C and γ by Cross-Validation	52
9.5	Multi-Class SVM	53
9.5.1	One-vs-One (OvO)	53
9.5.2	One-vs-All (OvA)	53
9.6	SVM vs. Other Classifiers	53
10	Chapter 10 – Deep Learning and Neural Networks	55
10.1	From Linear to Non-Linear: The Need for Activation Functions	55
10.2	Architecture of a Multilayer Perceptron (MLP)	55
10.3	Activation Functions	56
10.3.1	Why ReLU is Preferred in Hidden Layers	56
10.4	Loss Functions	56
10.5	Training: Backpropagation and Gradient Descent	57
10.5.1	The Gradient Descent Update Rule	57
10.5.2	Backpropagation	57
10.5.3	Mini-Batch Stochastic Gradient Descent	57
10.5.4	Adaptive Optimisers	57
10.6	Hyperparameters of Neural Networks	58
10.7	Regularisation of Neural Networks	58
10.7.1	L2 Regularisation (Weight Decay)	58
10.7.2	Dropout	58
10.7.3	Early Stopping	59
10.8	The Bias-Variance Trade-off in Neural Networks	59
10.9	Neural Networks vs. Other Methods	60
11	Chapter 11 – Unsupervised Learning	62
11.1	Principal Component Analysis	62
11.1.1	Motivation and Intuition	62
11.1.2	Loading Vectors and Scores	62
11.1.3	Proportion of Variance Explained	62
11.1.4	Choosing the Number of Components	63
11.1.5	The Biplot	63
11.1.6	PCA for Regression: Principal Components Regression	63
11.2	K -Means Clustering	63

11.2.1	Problem Formulation	63
11.2.2	The K -Means Algorithm	64
11.2.3	Choosing the Number of Clusters K	64
11.3	Hierarchical Clustering	65
11.3.1	Motivation	65
11.3.2	The Agglomerative Algorithm	65
11.3.3	Linkage Methods	65
11.3.4	Dissimilarity Measures	65
11.3.5	Dendrogram and Cluster Stability	66
11.4	Comparing Clustering Methods	66
11.5	Practical Considerations	66

Chapter 1 – Introduction to Statistical Learning

1.1 What is Statistical Learning?

Statistical learning refers to a vast collection of tools and methods for understanding and modelling data. At its core, statistical learning is about estimating a function f that relates a set of input variables (predictors, features) $X = (X_1, X_2, \dots, X_p)$ to an output variable (response) Y . The general relationship is written as:

$$Y = f(X) + \varepsilon$$

Here, f is a fixed but unknown function capturing the systematic relationship between the inputs and the output. The term ε is a random error term, independent of X , with $\mathbb{E}[\varepsilon] = 0$. Statistical learning is fundamentally concerned with estimating f from data.

1.2 Notation

The following notation is used throughout the compendium and corresponds directly to ISLP:

Symbol	Meaning
n	Number of observations (rows) in the dataset
p	Number of predictors (features/columns)
x_{ij}	Value of the j -th predictor for the i -th observation
$\mathbf{X}$	$n \times p$ data matrix of all predictor values
y_i	Value of the response variable for observation i
Y	The response variable (what we want to predict or explain)
$\hat{f}$	Our estimated model or function
$\hat{y}_i$	Fitted/predicted value for observation i : $\hat{y}_i = \hat{f}(x_i)$
ε	Random error term: $\mathbb{E}[\varepsilon] = 0$, independent of X
β_j	Regression coefficient for predictor j
$\hat{\beta}_j$	Estimated regression coefficient for predictor j

1.3 Why Do We Estimate f ?

There are two main reasons for estimating f : **prediction** and **inference**. These represent fundamentally different goals that affect how we choose and interpret models.

1.3.1 Prediction

In a prediction setting, we have access to a set of inputs X , but the response Y is not directly available or is costly to obtain. We use our estimate $\hat{f}$ to predict:

$$\hat{Y} = \hat{f}(X)$$

In this setting, we typically treat $\hat{f}$ as a **black box**: we care only about how accurate $\hat{Y}$ is as a prediction for Y , not about the internal workings of $\hat{f}$. The accuracy of $\hat{Y}$ depends on two sources of error:

- **Reducible error**: The gap between $\hat{f}$ and the true f . This error can, in principle, be reduced by choosing a better estimation method.
- **Irreducible error**: The variability in Y due to ε . Even if we knew f perfectly, this error would remain because $Y = f(X) + \varepsilon$ and ε contains unknown influences not captured by X . No model, however sophisticated, can eliminate this error.

The expected mean squared prediction error for a new observation at x_0 can be decomposed as:

$$\mathbb{E}[(Y - \hat{f}(x_0))^2] = \underbrace{[f(x_0) - \hat{f}(x_0)]^2}_{\text{reducible}} + \underbrace{\text{Var}(\varepsilon)}_{\text{irreducible}}$$

1.3.2 Inference

In an inference setting, we are interested in understanding the relationship between X and Y — specifically *how* Y changes as individual predictors change. Key inferential questions include:

- Which predictors are associated with the response?
- What is the direction and magnitude of each predictor's effect?
- Is the relationship linear, or is it more complex?

In inference, interpretability matters greatly. A linear model where each coefficient has a clear meaning is often preferred over a highly flexible black-box model, even if the latter predicts slightly better.

Prediction vs. Inference: A Key Distinction

Prediction: We want $\hat{Y}$ to be close to Y . The form of $\hat{f}$ does not matter — only its accuracy. Black-box methods (random forests, neural networks) are often preferred.

Inference: We want to understand how Y changes with X . The form of $\hat{f}$ must be interpretable. Simpler models (OLS, logistic regression) are preferred.

Many real problems involve both goals simultaneously. The choice of method should reflect the primary objective.

1.4 Parametric vs. Non-Parametric Methods

Statistical learning methods fall into two broad families based on how they make assumptions about the shape of f :

1.4.1 Parametric Methods

Parametric methods assume a specific functional form for f before looking at the data. The estimation process then reduces to estimating a finite set of parameters.

Example: Linear regression assumes $f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$. We estimate the $(p + 1)$ parameters $\beta_0, \beta_1, \dots, \beta_p$ using the training data.

Advantages: - Computationally simple and fast. - Interpretable coefficients and statistical tests. - Works well with limited data if the assumed form is approximately correct.

Disadvantages: - If the assumed form does not match the true f , the model will be misspecified and produce poor estimates. - Adding many parameters to increase flexibility risks **overfitting** — fitting the noise in the training data rather than the true signal.

1.4.2 Non-Parametric Methods

Non-parametric methods make no explicit assumption about the functional form of f . They allow f to take any shape that closely follows the observed data.

Examples: K-nearest neighbours, smoothing splines, local regression, kernel methods.

Advantages: - Much more flexible; can capture complex non-linear relationships. - Do not suffer from model misspecification.

Disadvantages: - Require far more data to produce accurate estimates, because no structural assumptions reduce the estimation problem. - Often harder to interpret — there are no coefficients to read off. - Risk of overfitting if not carefully regularised.

	Parametric	Non-Parametric
Assumes form of f?	Yes (e.g. linear)	No
Estimation	Fit parameters	Fit f directly to data
Data require-ment	Low	High
Interpretability	High	Low

	Parametric	Non-Parametric
Risk of misspecification	Yes	No
Risk of overfitting	With many parameters	Without regularisation

1.5 Supervised vs. Unsupervised Learning

Supervised learning is the setting where both predictor variables X and a response variable Y are observed. The goal is to learn the mapping from X to Y :

- If Y is **quantitative** (numerical): this is a *regression* problem.
- If Y is **qualitative** (categorical): this is a *classification* problem.

Unsupervised learning is the setting where we observe only X , with no response variable Y . The goal is to discover interesting structure or patterns in X itself. Common tasks include:

- **Clustering**: group observations into clusters of similar observations.
- **Dimensionality reduction**: find a low-dimensional representation of X that preserves most of the information (e.g. PCA).

Unsupervised learning is more exploratory and harder to evaluate (there is no “correct answer” to compare predictions against).

The Prediction–Interpretability Trade-off

Flexible methods (e.g. neural networks, random forests) can capture very complex relationships and tend to predict well, but are difficult to interpret.

Restrictive methods (e.g. linear regression) are highly interpretable but may miss complex structure.

The right choice depends on whether the goal is **prediction** (favour flexibility) or **inference** (favour interpretability). There is no universally best method — this is sometimes called the **No Free Lunch theorem**.

Theory Questions – Chapter 1

Q1: What is the difference between reducible and irreducible error? Can we, in principle, eliminate both by choosing a better model?

A: Reducible error arises because our estimate $\hat{f}$ is not a perfect approximation of the true function f . In principle, this error can be reduced by choosing a more appropriate statistical learning method or by collecting more data. Irreducible error, on the other hand, arises from ε — the random variation in Y that cannot be explained by X , no matter how well we know f . This includes unmeasured variables, inherent randomness in the process, and measurement noise. No model, however complex, can eliminate irreducible error, because it is a fundamental property of the data-generating process, not of our modelling choices.

Q2: When would you prefer a more restrictive, less flexible model over a highly flexible one?

A: A more restrictive model is preferred in two main situations. First, when the goal is **inference**: if we want to understand how individual predictors relate to the response — for example, “does advertising budget affect sales?” — a linear model with interpretable coefficients is far easier to reason about than a black-box neural network. Second, when data are **scarce**: flexible models have many effective parameters and require large samples to estimate them well. With small n , a flexible model will overfit to noise. A restrictive model that approximately captures the true f will generalise better in this situation.

Q3: Is it possible to have a problem that is simultaneously supervised and unsupervised?

A: In practice, some problems have elements of both. For example, we might use unsupervised methods

(like PCA) to reduce the dimensionality of X before applying a supervised model. However, supervised and unsupervised learning are conceptually distinct: supervised learning requires a labelled response Y , while unsupervised learning operates without Y . Semi-supervised learning, a related field, explicitly combines both by using a small labelled dataset alongside a large unlabelled one.

2 Chapter 2 – Assessing Model Accuracy

2.1 Measuring the Quality of Fit

After fitting a statistical model, we need to evaluate how well it performs. In the regression setting, the most commonly used measure is **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

MSE measures the average squared difference between the predicted values $\hat{f}(x_i)$ and the true values y_i . Smaller MSE means better fit.

Critically, we must distinguish between two types of MSE:

- **Training MSE**: computed using the same observations that were used to fit $\hat{f}$. A flexible model can always achieve very low training MSE by fitting the training data closely, including its noise — this is overfitting.
- **Test MSE**: computed using new observations not used in training. This is what we truly care about — how well does the model generalise to unseen data?

Training MSE is not enough

Training MSE always decreases (or stays flat) as the model becomes more flexible. This makes it a **poor** criterion for model selection. A model that memorises the training data will have very low training MSE but very high test MSE.

Always evaluate model performance on held-out test data. If test data are not available, use cross-validation (Chapter 5).

2.1.1 The Pattern of Training and Test MSE as Flexibility Increases

As model flexibility increases (e.g., increasing polynomial degree, decreasing regularisation penalty, growing a deeper tree):

- **Training MSE** monotonically decreases. The more flexible the model, the better it can fit the training data.
- **Test MSE** follows a characteristic **U-shape**: it initially decreases as the model captures true structure in the data, reaches a minimum at the optimal flexibility, then increases again as the model begins to overfit.

The minimum of the test MSE curve defines the **optimal level of flexibility** — the “sweet spot” between underfitting and overfitting.

2.2 The Bias-Variance Trade-off

The U-shape of test MSE is explained by the **bias-variance trade-off**, one of the most fundamental concepts in statistical learning. For any test point x_0 , the expected test MSE decomposes into exactly three components:

$$\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \underbrace{\text{Var}(\hat{f}(x_0))}_{\text{variance}} + \underbrace{[\text{Bias}(\hat{f}(x_0))]^2}_{\text{bias}^2} + \underbrace{\text{Var}(\varepsilon)}_{\text{irreducible error}}$$

Each component has a precise meaning:

Variance $\text{Var}(\hat{f}(x_0))$ refers to how much the estimated function $\hat{f}$ would change if we fitted it using a different training dataset. A high-variance model is very sensitive to the specific training data used — small changes in the training set lead to large changes in $\hat{f}$. Flexible models (deep trees, high-degree polynomials) tend to have high variance.

Bias $\text{Bias}(\hat{f}(x_0)) = f(x_0) - \mathbb{E}[\hat{f}(x_0)]$ refers to the error introduced by approximating the true, possibly complex function f with a simpler model $\hat{f}$. If the model is too rigid (e.g. fitting a straight line to clearly non-linear data), it will systematically miss the true pattern regardless of how much data we have. Restrictive models tend to have high bias.

Irreducible error $\text{Var}(\varepsilon)$ is the noise inherent in the data, unrelated to the model. It sets a lower bound on the achievable test MSE.

Component	High flexibility	Low flexibility
Variance	High — model changes a lot with data	Low — stable estimates
Bias	Low — can approximate complex f	High — too simple to capture f
Irreducible	Unchanged	Unchanged

The trade-off: Increasing model flexibility reduces bias but increases variance. Decreasing flexibility reduces variance but increases bias. The minimum test MSE occurs where the sum of squared bias and variance is minimised — the sweet spot. Importantly, **we can never reduce irreducible error**; it is always present regardless of our model.

Why does variance increase with flexibility?

Consider fitting a polynomial of degree 10 to 15 training observations. With so many parameters relative to observations, a small change in even one training point can lead to a dramatically different fitted curve. The model "memorises" specific data points rather than learning general patterns. This extreme sensitivity to the training data is what we mean by high variance.

Contrast this with a linear fit: regardless of which specific training observations we use, the fitted line will be broadly similar (similar slope, similar intercept), because the model is constrained to a simple form. It may be systematically wrong (bias), but it is consistently wrong.

2.3 The Classification Setting

So far we have discussed regression (quantitative Y). For **classification** (qualitative Y), the natural error measure is the **misclassification rate**:

$$\text{Error rate} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(y_i \neq \hat{y}_i)$$

where $\mathbf{1}(\cdot)$ is the indicator function (1 if the condition holds, 0 otherwise). As with MSE, we distinguish training error rate from test error rate, and the test error rate is what we care about.

The bias-variance trade-off applies in classification too: flexible models may overfit (low training error, high test error) and inflexible models may underfit (high error on both).

2.3.1 The Bayes Classifier

The theoretically optimal classifier — the one that minimises the expected test error rate — is the **Bayes classifier**. It classifies each observation to the class j with the highest posterior probability given the observed inputs:

$$\hat{y}(x_0) = \arg \max_j \Pr(Y = j \mid X = x_0)$$

The **Bayes error rate** is the lowest possible test error rate, achieved by the Bayes classifier:

$$\text{Bayes error rate} = 1 - \mathbb{E} \left[\max_j \Pr(Y = j \mid X) \right]$$

This is analogous to the irreducible error in regression. Even the Bayes classifier makes mistakes when the true class probabilities are not extreme (i.e., when $\Pr(Y = j \mid X)$ is not close to 0 or 1 for all classes). The Bayes error rate is the irreducible lower bound for any classifier.

The Bayes classifier is a theoretical benchmark only

In practice, we never know $\Pr(Y = j \mid X = x_0)$, so we cannot actually implement the Bayes classifier. It serves purely as a gold standard against which real classifiers are measured. Methods like LDA, logistic regression, and KNN can be viewed as different ways of estimating these posterior probabilities.

2.3.2 K-Nearest Neighbours Classifier

Since we cannot compute $\Pr(Y = j \mid X = x_0)$ directly, a simple approximation is to estimate it from the K nearest training observations. The **K-Nearest Neighbours (KNN) classifier** defines the neighbourhood $\mathcal{N}_0$ of x_0 as the set of K training points closest to x_0 (using Euclidean distance), and estimates the class probabilities as the proportion of neighbours from each class:

$$\Pr(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} \mathbf{1}(y_i = j)$$

The observation is then assigned to the class with the highest estimated probability.

The role of K : The choice of K directly controls the bias-variance trade-off in KNN:

K	Decision boundary	Bias	Variance	Behaviour
$K = 1$	Very jagged, irregular	Very low	Very high	Memorises training data; training error = 0
K moderate	Smooth, curved	Moderate	Moderate	Best test performance in practice
$K = n$	Flat (predicts majority class everywhere)	Very high	Zero	Ignores all local structure

Note that in KNN, increasing K **reduces** flexibility (increases bias, decreases variance), which is the opposite direction from, e.g., polynomial degree where increasing the degree increases flexibility. The optimal K is chosen by cross-validation.

Key insight: KNN approximates the Bayes classifier

The Bayes classifier assigns each observation to the class with highest true posterior probability $\Pr(Y = j \mid X = x_0)$.

KNN estimates this probability by: $\hat{P}(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} \mathbf{1}(y_i = j)$

As $n \rightarrow \infty$ and $K \rightarrow \infty$ such that $K/n \rightarrow 0$, KNN converges to the Bayes classifier. With finite data and optimal K , KNN can come surprisingly close to the Bayes error rate even when the true decision boundary is highly non-linear.

2.4 Summary: Key Principles of Model Assessment

The following principles underpin all model evaluation in statistical learning and will resurface throughout the compendium:

1. **Always evaluate on test data** (or use cross-validation). Training error is a misleading guide.
2. **The bias-variance trade-off** means there is no universally best level of model flexibility. The optimal model minimises expected test error.
3. **Irreducible error** sets a lower bound on achievable test error. A model that achieves the Bayes error rate (in classification) or $\text{Var}(\varepsilon)$ (in regression) is doing as well as theoretically possible.
4. **Model selection** should be based on estimated test error, not training error. Methods like cross-validation (Chapter 5) and information criteria (Chapter 6) are designed for this purpose.

Theory Questions – Chapter 2

Q1: Explain the bias-variance trade-off. Why does test MSE follow a U-shape as model flexibility increases?

A: The expected test MSE at any point x_0 decomposes into three terms: $\text{Var}(\hat{f}(x_0)) + \text{Bias}^2(\hat{f}(x_0)) + \text{Var}(\varepsilon)$. As model flexibility increases, bias decreases (the model can approximate more complex functions) but variance increases (the model becomes more sensitive to the specific training data). When flexibility is low, high bias dominates and test MSE is large. As flexibility increases, bias falls faster than variance rises, so test MSE decreases. Eventually, variance grows rapidly and begins to dominate, causing test MSE to rise again. This produces the characteristic U-shape. The minimum of the curve identifies the optimal flexibility. The irreducible error $\text{Var}(\varepsilon)$ is constant throughout and

represents the floor below which test MSE cannot fall.

Q2: What is the Bayes error rate and why is it important? Why can we not achieve zero error even with a perfect classifier?

A: The Bayes error rate is $1 - \mathbb{E}[\max_j \Pr(Y = j \mid X)]$ — the expected error of the theoretically optimal Bayes classifier, which always predicts the most probable class given X . It is the irreducible lower bound on the test error rate for any classifier: no method, regardless of complexity, can do better. The reason we cannot achieve zero error is that the classes may genuinely overlap in the feature space — i.e., two observations with identical X may belong to different classes. This overlap is due to unmeasured factors and inherent randomness (captured by ε), which no model can account for. The Bayes error rate quantifies exactly this irreducible confusion.

Q3: A model achieves training MSE = 0.5 and test MSE = 8.2. What does this tell you about the model, and what would you do?

A: The large gap between training MSE (0.5) and test MSE (8.2) is a classic sign of **overfitting** (high variance). The model has fitted the noise in the training data very closely — memorising it rather than learning the true underlying pattern — and therefore fails to generalise to new observations. To address this, one would reduce model complexity: for example, reduce polynomial degree, increase regularisation penalty (Ridge/Lasso), reduce tree depth, or use fewer features. Cross-validation (Chapter 5) should be used to select the appropriate level of complexity based on estimated test error.

3 Chapter 3 – Linear Regression

Linear regression is the most fundamental and widely used statistical learning method. Despite its simplicity, it forms the conceptual backbone for many more advanced methods (Ridge, Lasso, GLMs, GAMs). Understanding linear regression deeply — including its assumptions, diagnostics, and limitations — is essential for the exam.

3.1 Simple Linear Regression

Simple linear regression (SLR) models the relationship between a **single predictor** X and a quantitative response Y as a linear function:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Here, β_0 is the **intercept** (the expected value of Y when $X = 0$) and β_1 is the **slope** (the expected change in Y for a one-unit increase in X). The fitted model is:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

3.1.1 Estimating Coefficients — Ordinary Least Squares (OLS)

The principle of **Ordinary Least Squares** is to choose $\hat{\beta}_0$ and $\hat{\beta}_1$ so as to minimise the **Residual Sum of Squares (RSS)** — the total squared deviation between the observed and fitted values:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

Taking partial derivatives of RSS with respect to $\hat{\beta}_0$ and $\hat{\beta}_1$ and setting them to zero yields the closed-form OLS estimators:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

where $\bar{x} = \frac{1}{n} \sum x_i$ and $\bar{y} = \frac{1}{n} \sum y_i$ are the sample means.

Key properties of OLS estimators:

- They are **unbiased**: $\mathbb{E}[\hat{\beta}_0] = \beta_0$ and $\mathbb{E}[\hat{\beta}_1] = \beta_1$.
- By the **Gauss-Markov theorem**, OLS estimators are BLUE — **B**est **L**inear **U**nbiased **E**stimators — meaning that among all linear unbiased estimators, OLS has the smallest variance. This holds when errors are homoscedastic and uncorrelated.
- The fitted line always passes through the point $(\bar{x}, \bar{y})$.

3.1.2 Assessing the Accuracy of Coefficient Estimates

Because $\hat{\beta}_0$ and $\hat{\beta}_1$ are estimated from a sample, they are random variables with associated uncertainty. The **standard error** of $\hat{\beta}_1$ measures this uncertainty:

$$\text{SE}(\hat{\beta}_1) = \sqrt{\frac{\hat{\sigma}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}, \quad \text{SE}(\hat{\beta}_0) = \hat{\sigma} \sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

where $\hat{\sigma}^2 = \text{RSS}/(n - 2)$ is the estimated error variance (we divide by $n - 2$ because two parameters were estimated).

Interpretation of SE: A smaller SE means we have estimated the coefficient more precisely. SE decreases when:
 - n is larger (more data $\rightarrow$ more precise estimates).
 - $\text{Var}(X)$ is larger (more spread in $X \rightarrow$ better-identified slope).
 - The true error variance σ^2 is smaller (less noise in the data).

Hypothesis test for β_1 : We typically test $H_0 : \beta_1 = 0$ (no linear relationship between X and Y) against $H_1 : \beta_1 \neq 0$. The test statistic is:

$$t = \frac{\hat{\beta}_1}{\text{SE}(\hat{\beta}_1)}$$

Under H_0 , t follows a t -distribution with $n - 2$ degrees of freedom. A large $|t|$ (equivalently, a small p -value) gives evidence against H_0 .

A 95% confidence interval for β_1 is:

$$\hat{\beta}_1 \pm t_{\alpha/2, n-2} \cdot \text{SE}(\hat{\beta}_1)$$

This interval contains the true β_1 in approximately 95% of repeated samples. If the interval does not include zero, we reject $H_0 : \beta_1 = 0$ at the 5% level.

3.1.3 Assessing Model Accuracy: RSE, R^2 , and the F -statistic

Beyond the coefficients, we need measures of how well the overall model fits the data.

Residual Standard Error (RSE) estimates the standard deviation of ε — the average size of a residual:

$$\text{RSE} = \sqrt{\frac{\text{RSS}}{n - p - 1}}$$

where p is the number of predictors (so for SLR, $p = 1$ and $\text{RSE} = \sqrt{\text{RSS}/(n - 2)}$). RSE is on the same scale as Y — it tells us how far, on average, the model's predictions are from the true values.

R^2 (**coefficient of determination**) measures the proportion of total variance in Y explained by the model:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}, \quad \text{TSS} = \sum_{i=1}^n (y_i - \bar{y})^2$$

where TSS is the **Total Sum of Squares** (total variability in Y before fitting). $R^2 \in [0, 1]$: - $R^2 = 1$: perfect fit (all variation explained). - $R^2 = 0$: the model explains nothing beyond the mean.

Important: R^2 always increases (or stays flat) when we add more predictors, even if they are irrelevant. For multiple regression, **adjusted R^2** corrects for this (see Chapter 6).

In simple linear regression, $R^2 = \text{Cor}(X, Y)^2 = r^2$ — the squared correlation coefficient.

The F -statistic tests whether at least one predictor in the model is useful:

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0 \quad \text{vs.} \quad H_1 : \text{at least one } \beta_j \neq 0$$

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n - p - 1)}$$

Under H_0 , F follows an F -distribution with p and $n - p - 1$ degrees of freedom. A large F (small p -value) gives evidence that the model explains a meaningful amount of variation.

Key output statistics — summary table

Statistic	Formula	What it tells you
$\hat{\beta}_j$	OLS estimate	Expected ΔY per 1-unit $\uparrow X_j$, others fixed
$\text{SE}(\hat{\beta}_j)$	$\sqrt{\hat{\sigma}^2 / \sum (x_{ij} - \bar{x}_j)^2}$	Uncertainty in $\hat{\beta}_j$
t -statistic	$\hat{\beta}_j / \text{SE}(\hat{\beta}_j)$	Tests $H_0: \beta_j = 0$
p -value	$\Pr(T \geq t \mid H_0)$	$< 0.05 \rightarrow$ predictor significant
RSE	$\sqrt{\text{RSS} / (n - p - 1)}$	Avg. residual size, in units of Y
R^2	$1 - \text{RSS} / \text{TSS}$	Proportion of Y -variance explained
F -statistic	$\frac{(\text{TSS} - \text{RSS}) / p}{\text{RSS} / (n - p - 1)}$	Tests if any predictor is useful

3.1.4 Confidence Intervals vs. Prediction Intervals

After fitting a model, we often want to make statements about Y at a new input value x_0 . Two types of intervals serve different purposes:

Confidence interval (CI) for the *mean response* at x_0 , i.e., for $\mathbb{E}[Y \mid X = x_0] = \beta_0 + \beta_1 x_0$:

$$\hat{y}(x_0) \pm t_{\alpha/2, n-2} \cdot \text{SE}(\hat{y}(x_0))$$

This expresses our uncertainty about where the true regression line lies — uncertainty that shrinks as n grows.

Prediction interval (PI) for a *single new observation* y_0 at x_0 :

$$\hat{y}(x_0) \pm t_{\alpha/2, n-2} \cdot \sqrt{\text{SE}(\hat{y}(x_0))^2 + \hat{\sigma}^2}$$

The PI is always **wider** than the CI because it must account for two sources of uncertainty: 1. Uncertainty about the true mean $\mathbb{E}[Y \mid X = x_0]$ (same as CI). 2. The irreducible variance $\hat{\sigma}^2$ — even if we knew the true line exactly, individual observations scatter around it.

CI vs. PI: A practical distinction

CI: "We are 95% confident that the *average* sales for stores with 1000 customers lies between 40 and 60 units."

PI: "We are 95% confident that *this specific store* with 1000 customers will sell between 25 and 75 units."

The PI is wider because even after perfectly estimating the mean response, individual observations still vary around the mean. The PI uncertainty never vanishes even with infinite data (irreducible error), whereas the CI width $\rightarrow 0$ as $n \rightarrow \infty$.

3.2 Multiple Linear Regression

When there are $p > 1$ predictors, the **multiple linear regression** model is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon$$

In matrix form: $\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, where $\mathbf{X}$ is the $n \times (p+1)$ design matrix (with a column of ones for the intercept) and $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top$.

The OLS solution minimises $\|\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}\|^2$ and has a closed-form solution:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$$

This requires $\mathbf{X}^\top \mathbf{X}$ to be invertible, which fails when predictors are perfectly collinear or when $p \geq n$.

3.2.1 Interpretation of Coefficients in Multiple Regression

$\hat{\beta}_j$ represents the expected change in Y for a one-unit increase in X_j , **holding all other predictors constant**. This “partial effect” interpretation is fundamentally different from simple regression.

Why coefficients change from simple to multiple regression: In simple regression, $\hat{\beta}_1$ captures the total association between X_1 and Y , including any association that runs through other variables correlated with X_1 . In multiple regression, $\hat{\beta}_1$ isolates the direct effect of X_1 on Y , controlling for the other predictors. This means:

- A predictor that appears significant in simple regression may become insignificant when others are included (confounding removed).
- A predictor that appears insignificant alone may become significant when confounders are controlled for (suppression removed).

3.2.2 Key Questions in Multiple Regression

Question	Answer
Is at least one predictor useful?	F -test: large F , small p -value
Which specific predictors are useful?	Individual t -tests; variable selection (Chapter 6)
How well does the model fit?	RSE (in units of Y) and $R^2 \in [0, 1]$
How accurate are predictions?	CI (mean response) or PI (new observation)

3.3 Extensions of the Linear Model

3.3.1 Interaction Terms

The standard linear model assumes that the effect of each predictor on Y does not depend on the values of other predictors — an **additivity** assumption. When this is violated, we include **interaction terms**:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \varepsilon$$

The effect of X_1 on Y is now $\beta_1 + \beta_3 X_2$, which depends on X_2 . This is an **interaction**: the effect of one predictor is moderated by the level of another.

The Hierarchy Principle

If an interaction term $X_1 X_2$ is included in the model, the main effects X_1 and X_2 must **always** be included too, even if their individual p -values are not significant.

Reason: The interpretation of the interaction coefficient $\hat{\beta}_3$ depends on the main effects being present. Removing a main effect while keeping its interaction changes the meaning of the other coefficients fundamentally.

3.3.2 Polynomial Regression

By including powers of a predictor as additional columns in the design matrix, we can capture non-linear relationships while still using OLS:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_d X^d + \varepsilon$$

This is still a linear model (linear in the parameters β_j), so all OLS theory applies. The polynomial degree d controls flexibility and is chosen via cross-validation. In practice, $d > 4$ is rarely used due to instability at the boundaries (Runge’s phenomenon).

3.3.3 Qualitative (Categorical) Predictors

A categorical variable with k levels cannot be entered into a regression as a single column of integers (that would impose an artificial ordering). Instead, it is encoded using $k - 1$ **dummy variables** (one level is omitted as the baseline/reference category):

$$\text{Example: } X = \{A, B, C\} \Rightarrow \text{create } D_B = \mathbf{1}(X = B), D_C = \mathbf{1}(X = C)$$

The omitted level (here A) is the **reference category**. The coefficients $\hat{\beta}_B$ and $\hat{\beta}_C$ measure the expected difference in Y relative to level A , holding other predictors fixed.

If all k dummies are included simultaneously, perfect multicollinearity arises (the dummy columns sum to the intercept column) — this is the **dummy variable trap**, and it makes $\mathbf{X}^\top \mathbf{X}$ singular and OLS undefined.

3.4 Potential Problems in Linear Regression

Fitting a linear model is only the beginning. It is essential to check whether the underlying assumptions are satisfied. Below are the most important diagnostic checks:

3.4.1 1. Non-linearity of the Relationship

OLS assumes a linear relationship between each X_j and Y . **Detection:** plot residuals $e_i = y_i - \hat{y}_i$ against fitted values $\hat{y}_i$. A curved pattern (U-shape or inverted U) indicates non-linearity.

Consequence: Systematic bias in predictions — the model is consistently wrong in certain regions of X .

Remedy: Add polynomial terms, splines, or use a non-linear method (Chapter 7).

3.4.2 2. Heteroscedasticity (Non-constant Error Variance)

OLS assumes $\text{Var}(\varepsilon_i) = \sigma^2$ for all i . When this fails, errors have different variances for different observations.

Detection: a “funnel” or “fan” shape in the residuals-vs-fitted plot (residuals spread wider for larger $\hat{y}$).

Consequence: Standard errors for coefficients are wrong, making t -tests and CIs invalid.

Remedy: Transform Y (e.g., $\log Y$), use weighted least squares, or use robust standard errors.

3.4.3 3. Correlated Errors (Autocorrelation)

OLS assumes errors ε_i are independent. In time series data, consecutive observations often have correlated errors.

Detection: Plot residuals against time/observation index; look for systematic patterns (runs of positive or negative residuals).

Consequence: Standard errors are underestimated — the model appears more precise than it is.

Remedy: Include time lags as predictors, use time-series models.

3.4.4 4. Outliers

An outlier is an observation with an unusually large residual — the observed y_i is far from $\hat{y}_i$.

Detection: Studentised residuals $r_i = e_i / \hat{\sigma}_{(-i)}$; values $|r_i| > 3$ are flagged as potential outliers.

Consequence: Outliers inflate RSE, reducing R^2 and inflating standard errors. They may or may not strongly affect coefficient estimates (depends on their leverage).

3.4.5 5. High-Leverage Points

A high-leverage point has an unusual value of X relative to the other observations — it is far from $\bar{x}$ in the predictor space. Unlike outliers (which concern y), leverage concerns only x .

Detection: The hat matrix diagonal h_{ii} (leverage): values $h_{ii} > 2(p+1)/n$ indicate high leverage.

Consequence: High-leverage points can have a disproportionate influence on the fitted regression line. A single high-leverage outlier can drastically change $\hat{\beta}$.

Distinction: An outlier has a large residual but may have low leverage (it lies at typical x -values but unusual y). A high-leverage point has unusual x -values but may not necessarily have a large residual (if it happens to fall on the regression line). The most dangerous combination is a high-leverage outlier.

3.4.6 6. Multicollinearity

Multicollinearity occurs when two or more predictors are highly correlated with each other.

Detection: - Pairwise correlation matrix (simple collinearity). - **Variance Inflation Factor (VIF)**: $VIF(\hat{\beta}_j) = \frac{1}{1-R_{X_j|X_{-j}}^2}$, where $R_{X_j|X_{-j}}^2$ is the R^2 from regressing X_j on all other predictors. $VIF > 5-10$ signals a problem.

Consequence: Individual coefficient estimates become very imprecise (large SE), even if the model as a whole fits well. The F -statistic may be significant (some combination of predictors is useful) while individual t -tests are not (we cannot isolate individual effects).

Remedy: Drop one of the correlated predictors, combine them (e.g., PCA), or use Ridge regression (Chapter 6), which is specifically designed to handle collinearity.

1. **Residuals vs. Fitted:** check for non-linearity (curved pattern) and heteroscedasticity (funnel shape)
2. **Q-Q plot of residuals:** check for normality of errors (needed for valid inference with small n)
3. **Residuals vs. Order:** check for autocorrelation (patterns over time)
4. **Studentised residuals:** identify outliers ($|r_i| > 3$)
5. **Leverage / hat values h_{ii} :** identify high-leverage points
6. **VIF:** detect multicollinearity ($> 5-10$ is problematic)

Problem	Detection	Main consequence	Remedy
Non-linearity	Curved residual plot	Biased predictions	Polynomials, splines, GAMs
Heteroscedasticity	Funnel residual plot	Invalid SEs and CIs	Log transform, WLS
Autocorrelation	Residuals vs. time	SEs too small	Lagged predictors, time-series models
Outliers	$ r_i > 3$	Inflated RSE	Investigate, possibly remove
High leverage	$h_{ii} > 2(p+1)/n$	Distorted $\hat{\beta}$	Investigate, possibly remove
Multicollinearity	$VIF > 10$	Large SEs for affected $\hat{\beta}_j$	Drop predictor, Ridge, PCA

3.5 KNN Regression vs. Linear Regression

An important comparison is between parametric linear regression and the non-parametric **K-Nearest Neighbours (KNN) regression**. KNN regression predicts the response at a new point x_0 as the average of the K nearest training observations:

$$\hat{f}(x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} y_i$$

where $\mathcal{N}_0$ is the set of K training points closest to x_0 .

Aspect	Linear Regression	KNN Regression
Functional form	Linear: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \dots$	None — adapts to local data
Interpretability	High: coefficients have direct meaning	Low: no global formula
Works best when	True relationship is approximately linear	True relationship is highly non-linear
With small n	Good (few parameters to estimate)	Poor (neighbours may not be truly “near”)
With large p	Handles well (coefficients for each predictor)	Degrades: curse of dimensionality
Extrapolation	Extrapolates beyond training range (linearly)	Cannot extrapolate: no nearby neighbours

The curse of dimensionality: In high dimensions (p large), all observations become approximately equidistant from any test point, so the K “nearest” neighbours are not actually close in feature space. KNN loses its local averaging advantage and may perform worse than a simple linear model. Linear regression does not suffer from this problem because it estimates global parameters rather than relying on local neighbourhoods.

Theory Questions – Chapter 3

Q1: What does the OLS estimator minimise, and what properties does it have? Under what conditions is OLS BLUE?

A: OLS minimises the Residual Sum of Squares: $RSS = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$. The resulting estimators $\hat{\beta}_0$ and $\hat{\beta}_1$ are unbiased ($\mathbb{E}[\hat{\beta}_j] = \beta_j$) and have a closed-form solution. By the Gauss-Markov theorem, OLS estimators are BLUE — Best Linear Unbiased Estimators — meaning that among all estimators that are both linear in $\mathbf{y}$ and unbiased, OLS achieves the smallest variance. The theorem requires four conditions: (1) the model is correctly specified (no omitted non-linearity), (2) errors have zero mean ($\mathbb{E}[\varepsilon] = 0$), (3) errors are homoscedastic ($\text{Var}(\varepsilon_i) = \sigma^2$ for all i), and (4) errors are uncorrelated ($\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ for $i \neq j$). Note that normality of errors is NOT required for Gauss-Markov; it is needed for exact inference (hypothesis tests and CIs) in small samples.

Q2: What is the difference between a confidence interval and a prediction interval for a new observation? Why is the prediction interval always wider?

A: Both intervals are centred at the same fitted value $\hat{y}(x_0)$, but they capture different quantities and therefore have different widths. A **confidence interval** expresses uncertainty about the true mean response $\mathbb{E}[Y | X = x_0] = \beta_0 + \beta_1 x_0$ — it quantifies how precisely we have estimated the regression line at x_0 . A **prediction interval** expresses uncertainty about a specific new observation y_0 at x_0 . The PI is wider because it accounts for two independent sources of uncertainty: (1) the same uncertainty in the estimated regression line (as in the CI), and (2) the irreducible error $\hat{\sigma}^2$ from the fact that even if we knew the true line, a new observation would still randomly deviate from it. As $n \rightarrow \infty$, the CI width shrinks to zero (we estimate the line perfectly), but the PI width converges to $\pm t_{\alpha/2} \cdot \sigma$ — it never vanishes because irreducible error remains.

Q3: In a multiple regression, a predictor has a large t -statistic and small p -value, yet the F -statistic for the overall model is not significant. How can this happen? What is the reverse situation?

A: This situation can arise when there are many predictors (p is large relative to n). With many predictors, the overall F -test has low power because the model has many degrees of freedom to estimate. However, if one predictor happens to be strongly correlated with Y by chance, its individual t -statistic may be large. The reverse situation — significant F but no significant individual t -statistics — is more common and arises from **multicollinearity**: when predictors are highly correlated, no individual predictor can be identified as uniquely responsible for explaining Y , so all individual t -statistics are small. Yet, collectively the predictors do explain Y well, giving a large F . This is why one should check both the F -statistic (overall model utility) and individual t -statistics (individual predictor utility), and always check VIFs for multicollinearity.

4 Chapter 4 – Classification

Classification is the task of predicting a qualitative (categorical) response Y . Common examples include predicting whether an email is spam or not, whether a patient has a disease, or whether a loan will default. The response Y takes values in a finite set of classes $\{1, 2, \dots, K\}$.

4.1 Why Not Linear Regression for Classification?

It might seem natural to encode the classes as numbers (e.g. $Y \in \{0, 1\}$ for binary classification) and fit a linear regression. This approach has two fundamental problems:

Problem 1 — Ordering: If Y has three classes encoded as $\{0, 1, 2\}$, linear regression implies class 2 is “twice as far” from class 0 as class 1 — an artificial ordering that may be meaningless for nominal classes like {stroke, drug overdose, epilepsy}.

Problem 2 — Out-of-range predictions: Linear regression produces predicted values $\hat{y} \in (-\infty, +\infty)$, while class probabilities must lie in $[0, 1]$. Predictions outside $[0, 1]$ are not interpretable as probabilities and can cause downstream problems.

For binary $Y \in \{0, 1\}$, linear regression can be used as a rough approximation (it estimates $\mathbb{E}[Y | X] = \Pr(Y = 1 | X)$), but the two problems above motivate a proper classification framework.

4.2 Logistic Regression

Logistic regression directly models the probability that Y belongs to a particular class. For binary classification with $Y \in \{0, 1\}$, we model $p(X) = \Pr(Y = 1 | X)$.

4.2.1 The Logistic Model

To ensure $p(X) \in [0, 1]$ for all values of X , we apply the **logistic (sigmoid) function** to a linear combination of the predictors:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

The sigmoid maps any real number to $(0, 1)$: as $\beta_0 + \beta_1 X \rightarrow +\infty$, $p(X) \rightarrow 1$; as $\beta_0 + \beta_1 X \rightarrow -\infty$, $p(X) \rightarrow 0$.

Rearranging, we obtain the **log-odds** (or **logit**) representation, which reveals that logistic regression is a linear model on the log-odds scale:

$$\underbrace{\frac{p(X)}{1 - p(X)}}_{\text{odds}} = e^{\beta_0 + \beta_1 X}$$

$$\underbrace{\log\left(\frac{p(X)}{1 - p(X)}\right)}_{\text{log-odds (logit)}} = \beta_0 + \beta_1 X$$

Interpreting β_1 :

- A one-unit increase in X multiplies the **odds** by e^{β_1} .
- The effect on the **probability** $p(X)$ is non-linear and depends on the current value of p : the change is largest when $p \approx 0.5$ and smallest near 0 or 1 (the sigmoid is flattest at the extremes).
- The **sign** of β_1 determines the direction: $\beta_1 > 0$ means $p(X)$ increases with X ; $\beta_1 < 0$ means it decreases.

Odds vs. probability

If the probability of success is $p = 0.75$, then the odds are $p/(1 - p) = 0.75/0.25 = 3$ — meaning the event is three times as likely to happen as not to happen.

If $\beta_1 = 0.5$, then a one-unit increase in X multiplies the odds by $e^{0.5} \approx 1.65$ — a 65% increase in the odds. This multiplicative interpretation holds regardless of the current value of X , whereas the effect on the probability itself is non-linear and context-dependent.

4.2.2 Estimating Coefficients — Maximum Likelihood

Unlike OLS for linear regression, there is no closed-form solution for the logistic regression coefficients. Instead, they are estimated by **Maximum Likelihood Estimation (MLE)**: we find the values of β that maximise the probability of observing the data we actually observed.

The log-likelihood function is:

$$\ell(\beta) = \sum_{i: y_i=1} \log p(x_i) + \sum_{i: y_i=0} \log(1 - p(x_i))$$

This is also called the **binary cross-entropy loss** (negated). Maximising $\ell(\beta)$ — equivalently, minimising the cross-entropy — is done numerically using iterative algorithms such as **Newton-Raphson** (or IRLS: Iteratively Reweighted Least Squares). The algorithm is guaranteed to converge to the global maximum because the log-likelihood is concave.

The resulting MLE estimators $\hat{\beta}$ are: - **Consistent**: converge to the true β as $n \rightarrow \infty$. - **Asymptotically normal**: for large n , $\hat{\beta}_j \sim \mathcal{N}(\beta_j, \text{SE}(\hat{\beta}_j)^2)$, enabling z -tests and CIs. - **Not unbiased** in finite samples (unlike OLS), but the bias is small for moderate n .

4.2.3 Multiple Logistic Regression

With p predictors, the model extends naturally:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p}}$$

Each $\hat{\beta}_j$ is interpreted as the change in log-odds of $Y = 1$ for a one-unit increase in X_j , **holding all other predictors fixed**. All the warnings about multiple regression (confounding, collinearity, etc.) apply here too.

4.2.4 Multinomial Logistic Regression ($K > 2$ Classes)

For $K > 2$ classes, we choose one class as the **baseline** (say class K) and model $K - 1$ log-odds ratios relative to it. For each class $k = 1, \dots, K - 1$:

$$\log\left(\frac{\Pr(Y = k | X)}{\Pr(Y = K | X)}\right) = \beta_{k0} + \beta_{k1} X_1 + \cdots + \beta_{kp} X_p$$

Solving for the probabilities:

$$\Pr(Y = k | X) = \frac{e^{\beta_{k0} + \beta_k^\top \mathbf{X}}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_l^\top \mathbf{X}}}, \quad \Pr(Y = K | X) = \frac{1}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_l^\top \mathbf{X}}}$$

All K probabilities sum to 1. The interpretation of $\hat{\beta}_{kj}$ is: a one-unit increase in X_j is associated with a change in the log-odds of class k vs. baseline class K of $\hat{\beta}_{kj}$, holding other predictors fixed.

4.3 Evaluating Classifiers

4.3.1 The Confusion Matrix

A confusion matrix tabulates predicted classes against actual classes. For binary classification:

	Predicted: Negative ($\hat{Y} = 0$)	Predicted: Positive ($\hat{Y} = 1$)
Actual: Negative ($Y = 0$)	True Negative (TN)	False Positive (FP)
Actual: Positive ($Y = 1$)	False Negative (FN)	True Positive (TP)

From this table, a rich set of performance metrics can be derived:

Metric	Formula	Interpretation
Sensitivity / Recall / TPR	$TP/(TP + FN)$	Proportion of actual positives correctly identified
Specificity / TNR	$TN/(TN + FP)$	Proportion of actual negatives correctly identified
Precision / PPV	$TP/(TP + FP)$	Proportion of predicted positives that are truly positive
Overall Accuracy	$(TP + TN)/n$	Proportion of all observations correctly classified
False Positive Rate (FPR)	$FP/(FP + TN)$	$= 1 - \text{Specificity}$; rate of false alarms
F1-score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall; useful when classes are imbalanced

Accuracy is misleading for imbalanced classes

If 99% of observations belong to class 0, a classifier that always predicts 0 achieves 99% accuracy — yet it is completely useless. In such settings, sensitivity and precision are far more informative. Always report the full confusion matrix and multiple metrics when class imbalance is present.

4.3.2 The ROC Curve and AUC

Most classifiers produce a continuous score (e.g. $\hat{p}(X)$ from logistic regression) that is then thresholded to produce a class label. The default threshold is 0.5, but this is not always optimal.

The **Receiver Operating Characteristic (ROC) curve** visualises classifier performance across all possible thresholds by plotting: - y -axis: **True Positive Rate** (Sensitivity) $= TP/(TP + FN)$ - x -axis: **False Positive Rate** $= FP/(FP + TN) = 1 - \text{Specificity}$

As the threshold decreases from 1 to 0, more observations are classified as positive, so both TPR and FPR increase. The ROC curve traces this trade-off.

Key reference points: - A **perfect classifier** passes through the top-left corner (0, 1): $TPR = 1$, $FPR = 0$. - A **random classifier** lies on the diagonal from (0, 0) to (1, 1): it has no discriminative ability. - A classifier **below the diagonal** is worse than random (flip all predictions to improve it).

Area Under the Curve (AUC): the AUC summarises classifier performance across all thresholds in a single number:

$$AUC = \Pr(\hat{p}(\text{random positive}) > \hat{p}(\text{random negative}))$$

That is, AUC is the probability that the model ranks a randomly chosen positive observation higher than a randomly chosen negative one. $AUC \in [0.5, 1]$, with $AUC = 1$ indicating a perfect classifier and $AUC = 0.5$ indicating random guessing.

4.3.3 Cost-Optimal Classification Threshold

The default threshold of 0.5 is optimal only when the costs of a false positive and a false negative are equal. In practice, these costs are often asymmetric — for example, missing a cancer diagnosis (FN) is typically far more costly than an unnecessary follow-up test (FP).

The **cost-minimising threshold** is derived by comparing the expected cost of predicting each class. Predict positive if:

$$\hat{p}(X) \cdot c_{FN} > (1 - \hat{p}(X)) \cdot c_{FP}$$

Solving for $\hat{p}(X)$ gives the optimal threshold:

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}}$$

where c_{FP} is the cost of a false positive and c_{FN} is the cost of a false negative.

Cost-threshold intuition

- $c_{FN} \gg c_{FP}$ (missing a positive is very costly): $t^* \ll 0.5$ — lower the threshold to catch more positives, accepting more false alarms.
- $c_{FP} \gg c_{FN}$ (false alarms are very costly): $t^* \gg 0.5$ — raise the threshold, only predict positive when very confident.
- $c_{FN} = c_{FP}$: $t^* = 0.5$ — the symmetric default is optimal.

Total expected cost = $FN \times c_{FN} + FP \times c_{FP}$. To find the optimal threshold in practice, evaluate total cost across a grid of thresholds on validation data.

4.4 Generative Models for Classification

An alternative to directly modelling $\Pr(Y = k | X)$ (the **discriminative** approach of logistic regression) is the **generative** approach: model the distribution of X within each class $\Pr(X | Y = k)$, then apply Bayes' theorem to obtain the posterior:

$$\Pr(Y = k | X = x) = \frac{\pi_k \cdot f_k(x)}{\sum_{l=1}^K \pi_l \cdot f_l(x)}$$

where: - $\pi_k = \Pr(Y = k)$ is the **prior** probability of class k (estimated as $\hat{\pi}_k = n_k/n$). - $f_k(x) = \Pr(X = x | Y = k)$ is the **class-conditional density** (what X looks like within class k).

The different generative classifiers (LDA, QDA, Naive Bayes) differ in how they model $f_k(x)$.

4.5 Linear Discriminant Analysis (LDA)

LDA assumes that within each class, X follows a **multivariate Gaussian distribution** with class-specific mean vectors μ_k but a **shared covariance matrix** Σ (the same for all classes):

$$X | Y = k \sim \mathcal{N}(\mu_k, \Sigma)$$

4.5.1 LDA for $p = 1$ (Single Predictor)

With one predictor, the class-conditional density is:

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma^2}\right)$$

Plugging into Bayes' theorem and taking logarithms, one can show that the posterior is maximised by assigning x to the class with the largest **linear discriminant function**:

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log \pi_k$$

This is **linear in x** (hence “Linear” DA), meaning the decision boundary between any two classes is a single point on the real line. For two classes with equal priors, the boundary is simply the midpoint $(\mu_1 + \mu_2)/2$.

Parameter estimates from training data:

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i: y_i=k} x_i, \quad \hat{\sigma}^2 = \frac{1}{n-K} \sum_k \sum_{i: y_i=k} (x_i - \hat{\mu}_k)^2$$

4.5.2 LDA for $p > 1$ (Multiple Predictors)

With p predictors, the multivariate discriminant function is:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k$$

This is again linear in $\mathbf{x}$, so the decision boundaries between classes are **hyperplanes** (linear in the feature space). The estimated parameters are:

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i: y_i=k} \mathbf{x}_i, \quad \hat{\boldsymbol{\Sigma}} = \frac{1}{n-K} \sum_{k=1}^K \sum_{i: y_i=k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top$$

The key requirement is that $\hat{\boldsymbol{\Sigma}}$ is invertible, which requires $n > p + K$.

Why does a shared $\boldsymbol{\Sigma}$ lead to linear boundaries?

The log-posterior ratio between class k and class l is:

$$\log \frac{\Pr(Y = k | \mathbf{x})}{\Pr(Y = l | \mathbf{x})} = \log \frac{\pi_k}{\pi_l} + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_l) - \frac{1}{2}(\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k - \boldsymbol{\mu}_l^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_l)$$

Because $\boldsymbol{\Sigma}$ is the same for both classes, the quadratic terms in $\mathbf{x}$ cancel, leaving a linear function of $\mathbf{x}$. Setting this equal to zero gives the linear decision boundary. If $\boldsymbol{\Sigma}_k \neq \boldsymbol{\Sigma}_l$ (as in QDA), the quadratic terms do not cancel and the boundary becomes quadratic.

4.6 Quadratic Discriminant Analysis (QDA)

QDA relaxes the shared-covariance assumption and allows each class to have its **own covariance matrix** $\boldsymbol{\Sigma}_k$:

$$X | Y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

The discriminant function becomes:

$$\delta_k(\mathbf{x}) = -\frac{1}{2} \log |\boldsymbol{\Sigma}_k| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k$$

This is **quadratic in $\mathbf{x}$** , giving curved (quadratic) decision boundaries. QDA must estimate K separate covariance matrices $\boldsymbol{\Sigma}_k$, each of size $p \times p$ — a total of $Kp(p+1)/2$ parameters. This makes QDA much more flexible but also more data-hungry than LDA.

4.6.1 LDA vs. QDA: When to Use Which

Criterion	LDA	QDA
Covariance assumption	Shared $\boldsymbol{\Sigma}$ across all classes	Class-specific $\boldsymbol{\Sigma}_k$
Decision boundary	Linear (hyperplane)	Quadratic (curved surface)
Number of parameters	$Kp + p(p+1)/2$	$Kp + Kp(p+1)/2$
Bias	Higher (more constrained)	Lower
Variance	Lower	Higher
Preferred when	n is small relative to p ; classes have similar spread	n is large; classes have clearly different covariances
True boundary is linear	LDA is better or equal	QDA may overfit
True boundary is curved	LDA may underfit	QDA is better

The LDA–QDA choice is another instance of the bias-variance trade-off: LDA makes a stronger assumption (same $\boldsymbol{\Sigma}$) that reduces variance but increases bias if wrong; QDA is more flexible but requires more data.

4.7 Naive Bayes

Naive Bayes is a generative classifier that makes an even stronger simplifying assumption: **within each class, the predictors are mutually independent**:

$$f_k(x) = \prod_{j=1}^p f_{kj}(x_j)$$

where f_{kj} is the marginal density of the j -th predictor within class k . This drastically reduces the number of parameters to estimate: instead of a full $p \times p$ covariance matrix, we need only p univariate densities per class.

How each f_{kj} is modelled: - If X_j is quantitative: assume Gaussian, estimate $\hat{\mu}_{kj}$ and $\hat{\sigma}_{kj}^2$ from class- k observations. - If X_j is categorical: estimate as the proportion of each category within class k .

The conditional independence assumption is almost certainly wrong in practice, but Naive Bayes often works surprisingly well because: 1. Even a misspecified model can produce good class rankings. 2. When p is large and n is small, the variance reduction from the independence assumption outweighs the bias it introduces — the model generalises better despite being “wrong.”

Naive Bayes is particularly popular for text classification (where p can be tens of thousands of words) precisely because the independence assumption keeps the model tractable.

4.8 K-Nearest Neighbours Classifier (Revisited)

KNN classification (introduced in Chapter 2) estimates the posterior class probabilities non-parametrically using the K closest training observations:

$$\Pr(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} \mathbf{1}(y_i = j), \quad \hat{y} = \arg \max_j \Pr(Y = j \mid X = x_0)$$

KNN makes no distributional assumptions and can produce arbitrarily complex, non-linear decision boundaries. However: - **Standardise** all predictors before applying KNN (Euclidean distance is scale-dependent). - KNN degrades in high dimensions (curse of dimensionality). - There is no interpretable model — just a look-up of neighbours. - Choose K by cross-validation.

4.9 Comparison of Classification Methods

Method	Decision boundary	Key assumption	Best when
Logistic regression	Linear	Linear log-odds	Binary/multinomial Y ; interpretability needed; no strong distributional assumption
LDA	Linear	Gaussian $X \mid Y = k$; shared Σ	Small n ; $K > 2$ classes; Gaussian predictors with similar spread
QDA	Quadratic	Gaussian $X \mid Y = k$; class-specific Σ_k	Large n ; classes have clearly different covariance structures
Naive Bayes	Depends on f_{kj}	Conditional independence within each class	Very large p , small n ; text/document classification
KNN	Arbitrary non-linear	Proximity in X -space implies similar Y	Highly non-linear boundary; large n ; small p

LDA vs. Logistic Regression: a subtle but important distinction

Both LDA and logistic regression produce **linear** decision boundaries, and their performance is often similar in practice. The key difference is in how they estimate the boundary:

- **Logistic regression** is *discriminative*: it directly models $\Pr(Y = 1 \mid X)$ without any assumption on $\Pr(X)$.

- **LDA** is *generative*: it models $\Pr(X | Y = k)$ as Gaussian and derives $\Pr(Y = k | X)$ via Bayes' theorem.

When the Gaussian assumption holds, LDA uses the distributional information about X more efficiently and can outperform logistic regression (especially when n is small). When the assumption fails badly, logistic regression is more robust. In practice, both should be tried.

Theory Questions – Chapter 4

Q1: Why can we not use linear regression for a classification problem with two classes, even if we encode them as 0 and 1? What does logistic regression model instead, and what function ensures predictions stay in $[0, 1]$?

A: Linear regression is inappropriate for binary classification for two reasons. First, it can produce fitted values outside $[0, 1]$, which are not interpretable as class probabilities. Second, with more than two classes, numerically encoding the classes (e.g. $\{0, 1, 2\}$ for three classes) imposes an artificial ordering and equal spacing that is meaningless for nominal categories. Logistic regression instead directly models the probability $p(X) = \Pr(Y = 1 | X)$ by applying the **sigmoid (logistic) function** to a linear combination of the predictors: $p(X) = e^{\beta_0 + \beta_1 X} / (1 + e^{\beta_0 + \beta_1 X})$. The sigmoid has range $(0, 1)$ for all real inputs, guaranteeing that predicted probabilities are always valid. The model is linear on the **log-odds (logit) scale**: $\log(p/(1 - p)) = \beta_0 + \beta_1 X$.

Q2: What is the AUC of the ROC curve, and what does an AUC of 0.5 vs. 0.9 tell you about a classifier?

A: The AUC (Area Under the ROC Curve) is the probability that the model assigns a higher predicted probability to a randomly chosen positive observation than to a randomly chosen negative one: $\text{AUC} = \Pr(\hat{p}(\text{positive}) > \hat{p}(\text{negative}))$. The ROC curve plots TPR (sensitivity) against FPR ($1 - \text{specificity}$) as the classification threshold varies from 1 to 0. An AUC of 0.5 means the classifier has no discriminative ability whatsoever — it performs no better than randomly assigning classes (this corresponds to the diagonal of the ROC plot). An AUC of 0.9 means that in 90% of cases where we draw one random positive and one random negative observation, the model correctly ranks the positive higher. This is a strong classifier. The AUC is threshold-independent and particularly useful for comparing classifiers without committing to a specific decision threshold.

Q3: What is the fundamental difference between LDA and logistic regression, even though both produce linear decision boundaries? When would you prefer each?

A: Both methods produce linear decision boundaries and are often competitive in performance, but they differ fundamentally in their approach. Logistic regression is **discriminative**: it directly models $\Pr(Y = k | X)$ without making any assumptions about the distribution of X itself. LDA is **generative**: it models the class-conditional distributions $\Pr(X | Y = k)$ as multivariate Gaussians with a shared covariance matrix, then uses Bayes' theorem to derive $\Pr(Y = k | X)$. LDA is preferred when the Gaussian assumption approximately holds and n is small relative to p , because the additional structural information allows more efficient estimation. LDA also naturally extends to $K > 2$ classes without any modification. Logistic regression is preferred when the Gaussian assumption is clearly violated (e.g. binary or skewed predictors), when interpretability of log-odds coefficients is important, or when the primary goal is to model the class boundary directly without distributional assumptions.

5 Chapter 5 – Resampling Methods

Resampling methods are techniques that involve repeatedly drawing samples from training data — either to estimate how well a model will perform on new data (cross-validation) or to quantify uncertainty in an estimator (the bootstrap). They are among the most practically important tools in statistical learning, since analytical formulas for test error or standard errors often do not exist for complex models.

5.1 Cross-Validation

The central challenge in model assessment is that we want to know the **test error** — how well a model fitted on the training data performs on new, unseen observations. Cross-validation provides a principled way to estimate test error from training data alone, without needing a separate held-out test set.

5.1.1 The Validation Set Approach

The simplest approach: randomly split the available data into two halves — a **training set** and a **validation set** (also called a hold-out set). Fit the model on the training set and evaluate its error on the validation set.

$$\text{Validation MSE} = \frac{1}{|V|} \sum_{i \in V} (y_i - \hat{f}(x_i))^2$$

where V is the set of validation observations and $\hat{f}$ was fitted on the complementary training observations.

Advantages: Simple to implement; computationally cheap (fit the model only once).

Disadvantages: - **High variability:** the estimated test error depends strongly on which observations happen to fall in the training vs. validation set. Different random splits can give very different error estimates. - **Overestimates test error:** the model is trained on only $\approx 50\%$ of the data. A model trained on more data would perform better, so the validation error overestimates how poorly the model would perform if trained on the full dataset.

5.1.2 Leave-One-Out Cross-Validation (LOOCV)

LOOCV addresses both limitations of the validation set approach by using $n - 1$ observations for training and holding out exactly one observation at a time, repeating this for every observation:

1. For each $i = 1, 2, \dots, n$:
 - (a) Fit the model on all observations **except** i : $\hat{f}^{(-i)}$
 - (b) Predict observation i : $\hat{y}_i = \hat{f}^{(-i)}(x_i)$
 - (c) Compute the squared error: $\text{MSE}_i = (y_i - \hat{y}_i)^2$
2. Average over all n hold-outs:

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i$$

Advantages: - **Low bias:** the model is trained on $n - 1$ observations — almost the full dataset. The training set closely approximates what would be available in practice. - **Deterministic:** no random splitting, so the result is always the same (no variability from different random seeds).

Disadvantages: - **Expensive:** requires fitting the model n times. For large n or complex models (e.g. neural networks), this can be prohibitively slow. - **High variance:** the n training sets overlap almost completely (each differs from the others by only one observation), making the n error estimates highly correlated. The average of highly correlated estimates has higher variance than the average of independent estimates.

Computational shortcut for linear models: For OLS regression (and polynomial regression), LOOCV can be computed at the cost of a single model fit using the hat matrix shortcut:

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_{ii}} \right)^2$$

where h_{ii} is the i -th diagonal entry of the hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ (the **leverage** of observation i). This shortcut does not generalise to non-linear models.

5.1.3 k -Fold Cross-Validation

k -fold CV is the practical workhorse of model selection. The data are randomly divided into k roughly equal-sized **folds** (groups). For each fold $j = 1, \dots, k$:

1. Randomly partition the n observations into k folds of approximately equal size
2. For $j = 1, 2, \dots, k$:
 - (a) Hold out fold j as the test set
 - (b) Train the model on all remaining $k - 1$ folds
 - (c) Compute the test error on fold j : MSE_j
3. Average the k test errors:

$$\text{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^k \text{MSE}_j$$

LOOCV is the special case where $k = n$.

Typical choices: $k = 5$ or $k = 10$ are the standard recommendations in ISLP and the broader literature.

5.1.4 Bias-Variance Trade-off for k -Fold CV

The choice of k directly controls the bias-variance trade-off of the CV error estimate itself:

Method	Training set size	Bias of CV estimate	Variance of CV estimate	Computation
Validation set	$\approx n/2$	High (trained on half data)	High (sensitive to split)	1 fit
LOOCV ($k = n$)	$n - 1$	Very low	High (highly correlated folds)	n fits
$k = 10$ fold	$\approx 9n/10$	Low	Low-moderate	10 fits
$k = 5$ fold	$\approx 4n/5$	Low-moderate	Low	5 fits

The advantage of $k = 5$ or $k = 10$ over LOOCV is not just computational. Because the k training sets in k -fold CV are more different from each other (each fold contains n/k unique observations), the fold-level errors MSE_j are less correlated. The average of less correlated quantities has lower variance. Empirically, $k = 10$ has been shown to give test error estimates with low bias and low variance — a good balance.

Why does LOOCV have higher variance than k -fold despite training on more data?

It seems paradoxical that training on more data (LOOCV uses $n - 1$, k -fold uses $\approx (k - 1)n/k$) leads to higher variance in the error estimate. The key is that LOOCV produces n estimates $\text{MSE}_1, \dots, \text{MSE}_n$ that are almost perfectly correlated — each pair of training sets shares $n - 2$ identical observations. Highly correlated random variables have high variance when averaged. With k -fold CV, each pair of training sets shares only $(k - 2)n/k$ observations, so the fold estimates are less correlated, giving a lower-variance average.

5.1.5 CV for Model Selection

Cross-validation is most commonly used to **select among competing models** — choosing the polynomial degree, the regularisation parameter λ , or the number of neighbours K :

1. For each candidate model (or hyperparameter value), compute $\text{CV}_{(k)}$.
2. Select the model with the **lowest CV error** ($\lambda_{\min}$).
3. Optionally apply the **one-standard-error rule**: select the simplest model whose CV error is within one standard error of the minimum — often giving a sparser, more interpretable model with similar predictive performance.

Do not use the same data for both model selection and error reporting

If you select the best model by CV and then report that CV error as the test error, you are using the same data twice, which leads to optimistic (too low) error estimates. The correct approach is:

- (1) Use CV on the training set for model selection.
- (2) Report performance on a completely separate held-out test set (or use nested CV).

5.1.6 Cross-Validation for Classification

In classification problems, k -fold CV is applied identically, but MSE is replaced by the **misclassification rate** on each fold:

$$CV_{(k)} = \frac{1}{k} \sum_{j=1}^k \text{Err}_j, \quad \text{where } \text{Err}_j = \frac{1}{|V_j|} \sum_{i \in V_j} \mathbf{1}(y_i \neq \hat{y}_i)$$

Other metrics (AUC, F1) can also be averaged across folds.

5.2 The Bootstrap

The bootstrap is a general-purpose resampling tool for estimating the **uncertainty** (standard error, confidence interval) of virtually any statistic — including those for which no analytical formula exists.

5.2.1 Core Idea and Algorithm

The key insight is that the training dataset itself is our best approximation of the population. We can simulate “drawing new samples from the population” by repeatedly drawing **with replacement** from the training data.

1. Choose B (typically $B = 1000$ or more)
2. For $b = 1, 2, \dots, B$:
 - (a) Draw a bootstrap sample Z^{*b} of size n **with replacement** from the original dataset
 - (b) Compute the statistic of interest on Z^{*b} : call it $\hat{\theta}^{*b}$
3. Estimate the standard error:

$$\widehat{\text{SE}}_B(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}^{*b} - \bar{\hat{\theta}})^2}, \quad \text{where } \bar{\hat{\theta}} = \frac{1}{B} \sum_{b=1}^B \hat{\theta}^{*b}$$

4. Compute a 95% confidence interval using the **percentile method**:

$$95\% \text{ CI} = [2.5\text{th percentile of } \hat{\theta}^{*b}, 97.5\text{th percentile of } \hat{\theta}^{*b}]$$

The bootstrap distribution of $\hat{\theta}^{*b}$ around $\bar{\hat{\theta}}$ approximates the true sampling distribution of $\hat{\theta}$ around θ . This approximation improves as B increases and as n increases.

5.2.2 What Observations End Up in a Bootstrap Sample?

Since sampling is done **with replacement**, some observations will appear multiple times in a bootstrap sample and others will not appear at all. The probability that observation i is **not** selected in a single draw is $(1 - 1/n)$. Over n draws:

$$\Pr(\text{obs } i \notin Z^{*b}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0.368$$

So approximately **36.8% of observations are absent** from each bootstrap sample, and **63.2% are present** (some more than once). This fact will become important when we discuss Out-of-Bag error estimation for Random Forests (Chapter 8).

A key implication of the $\approx 36.8\%$ absent rate

Because each bootstrap sample leaves out about one third of the data, we could in principle evaluate model fit on the left-out observations — analogous to a test set. This is exactly the idea behind **Out-of-Bag (OOB) error** in bagging and random forests (Chapter 8). The OOB observations act as a built-in validation set for each bootstrap-fitted model, at no additional computational cost.

5.2.3 The Portfolio Example: When Bootstrap is Essential

Consider investing a fraction α in asset X and $(1 - \alpha)$ in asset Y , with the goal of minimising the variance of the total return. The variance-minimising allocation is:

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}$$

where $\hat{\sigma}_X^2$, $\hat{\sigma}_Y^2$ and $\hat{\sigma}_{XY}$ are the sample variances and covariance.

The problem: there is no simple closed-form formula for $\text{SE}(\hat{\alpha})$ — the formula involves complicated functions of sample moments. The bootstrap solves this effortlessly:

1. Draw $B = 1000$ bootstrap samples from the (X, Y) data.
2. Compute $\hat{\alpha}^{*b}$ for each bootstrap sample.
3. Report $\widehat{\text{SE}}_B(\hat{\alpha})$ as the bootstrap standard error.

This works for **any** statistic — the median, a correlation, the AUC of a classifier, a regularised regression coefficient, or any quantity for which you want to quantify uncertainty.

5.3 Cross-Validation vs. Bootstrap: A Critical Distinction

These two methods are complementary tools that answer different questions:

CV vs. Bootstrap — what each estimates

Cross-Validation estimates how well a *model* generalises to new data — it provides an estimate of **test error**. Use it to select among models or hyperparameters, or to report expected prediction performance.

Bootstrap estimates the *uncertainty* of a statistic or estimator — it provides **standard errors and confidence intervals**. Use it whenever you want to quantify how precisely you have estimated a quantity, especially when analytical formulas are unavailable.

They are not interchangeable: CV tells you about *model performance*; bootstrap tells you about *estimation uncertainty*.

Feature	Cross-Validation	Bootstrap
Primary purpose	Estimate test error / model selection	Estimate SE and CI of a statistic
Sampling scheme	Without replacement (disjoint folds)	With replacement
What varies across iterations	Which observations are held out	Which observations are included
Output	Estimated test MSE / error rate	$\widehat{\text{SE}}(\hat{\theta})$, confidence interval
Typical B or k	$k = 5$ or 10	$B = 1000+$
Requires model refitting	Yes, k times	Yes, B times

Theory Questions – Chapter 5

Q1: Explain the bias-variance trade-off in the choice of k in k -fold cross-validation. Why is $k = 5$ or $k = 10$ typically recommended over LOOCV?

A: The choice of k controls a bias-variance trade-off in the CV error estimate itself. With small k (e.g. $k = 2$), each model is trained on only half the data, so the training set is much smaller than the full dataset. This means each fold's model is weaker than the final model, and the CV error overestimates

the true test error (high bias). With $k = n$ (LOOCV), each model is trained on $n - 1$ observations, giving nearly unbiased estimates of test error (low bias). However, the n training sets in LOOCV are nearly identical (each differs by one observation), making the n fold-level error estimates highly correlated. The average of highly correlated estimates has higher variance than the average of weakly correlated estimates. $k = 5$ or $k = 10$ strikes a better balance: the training sets are different enough that fold errors are only moderately correlated (lower variance than LOOCV), yet large enough that bias is low. Empirically, these choices provide reliable test error estimates and are far cheaper than LOOCV for non-linear models.

Q2: In the bootstrap, approximately what fraction of observations are absent from each bootstrap sample? Why does this matter?

A: The probability that a specific observation is not selected in a bootstrap sample of size n drawn with replacement is $(1 - 1/n)^n$, which converges to $e^{-1} \approx 0.368$ as $n \rightarrow \infty$. So approximately 36.8% of observations are absent from each bootstrap sample and 63.2% are present (some appearing more than once). This matters for two reasons. First, it means that roughly one third of the data is always available as a "free" validation set for each bootstrap-fitted model — this is the basis of Out-of-Bag (OOB) error estimation in random forests, which gives a nearly unbiased estimate of test error without requiring cross-validation. Second, the duplicated observations in the bootstrap sample inflate apparent agreement (since the same observation may appear in both training and test sets in naive implementations), which is why bootstrap is not used as a direct substitute for CV in test error estimation.

Q3: A colleague proposes estimating the standard error of a regression coefficient by running cross-validation and computing the standard deviation of the coefficient estimates across folds. Is this a valid approach? What is the correct tool?

A: This is not a valid approach. Cross-validation is designed to estimate **test error** (model prediction performance), not to quantify the sampling variability of an estimator. The variation in a coefficient estimate across CV folds reflects the sensitivity of the model to which observations are included, but this is not the same as the standard error — the variation we would see if we repeatedly drew new datasets of size n from the true population. The correct tool is the **bootstrap**: draw B bootstrap samples from the training data, fit the model on each, and compute the coefficient estimate $\hat{\beta}_j^{*b}$ for each. The bootstrap standard error $\widehat{SE}_B(\hat{\beta}_j)$ directly approximates the true sampling variability. For OLS specifically, the analytical formula $SE(\hat{\beta}_j) = \hat{\sigma} \sqrt{[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}}$ is available and more efficient, but bootstrap remains useful for regularised estimators (Ridge, Lasso) where analytical SEs are not standard.

6 Chapter 6 – Linear Model Selection and Regularization

When the number of predictors p is large relative to the number of observations n , OLS suffers from two related problems. First, OLS has high variance: with many parameters to estimate from limited data, the coefficients are noisy and generalise poorly. Second, when $p \geq n$, OLS is entirely undefined because $\mathbf{X}^\top \mathbf{X}$ is singular. Three broad strategies address these problems: (1) **subset selection** — choose a subset of the p predictors; (2) **shrinkage/regularisation** — fit all predictors but shrink coefficients toward zero; and (3) **dimension reduction** — project the predictors onto a lower-dimensional space.

6.1 Subset Selection

6.1.1 Best Subset Selection

Fit a separate OLS model for every possible subset of the p predictors — a total of 2^p models. For each subset size $k = 0, 1, \dots, p$, identify the best model $\mathcal{M}_k$ (the one with the lowest RSS among all $\binom{p}{k}$ models of that size). Then choose the final model from $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$ using an appropriate criterion (CV, AIC, BIC, or adjusted R^2).

Limitation: Computationally infeasible for $p > 40$. With $p = 30$, there are $2^{30} \approx 10^9$ models to fit. This motivates stepwise alternatives.

6.1.2 Forward Stepwise Selection

Forward stepwise builds up the model from nothing, adding one predictor at a time:

1. Begin with $\mathcal{M}_0$: the null model containing only an intercept
 2. For $k = 0, 1, \dots, p - 1$:
 - (a) Consider all $p - k$ models that augment $\mathcal{M}_k$ with exactly one additional predictor
 - (b) Choose the one that gives the greatest improvement in RSS (or equivalently, R^2): call it $\mathcal{M}_{k+1}$
 3. Select the best overall model from $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$ using CV or an information criterion
- Total models considered:** $1 + p(p + 1)/2$ (vs. 2^p for best subset). Can be used even when $p > n$.

Key limitation: Forward stepwise is greedy — it adds the best predictor at each step but does not revisit earlier choices. A predictor added early may become redundant once others are included, but it cannot be removed. Therefore, forward stepwise is not guaranteed to find the globally optimal model of any given size.

6.1.3 Backward Stepwise Selection

Backward stepwise starts with the full model and removes predictors one at a time:

1. Begin with $\mathcal{M}_p$: the full model containing all p predictors
 2. For $k = p, p - 1, \dots, 1$:
 - (a) Consider all k models that drop exactly one predictor from $\mathcal{M}_k$
 - (b) Choose the one with the smallest decrease in RSS (i.e. drop the least useful predictor): call it $\mathcal{M}_{k-1}$
 3. Select the best overall model from $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$ using CV or an information criterion
- Requirement:** $n > p$ (the full model must be fittable). Same $O(p^2)$ cost as forward stepwise.

Forward and backward stepwise produce different sequences of models and may select different final models. Neither is guaranteed to match best subset selection, but both are computationally tractable for large p .

6.2 Choosing the Optimal Model Size

Once we have a sequence of models $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$, we must choose which one to use. We cannot use training RSS or R^2 directly, because these always improve as we add predictors — even irrelevant ones. Two approaches exist: **information criteria** (closed-form adjustments to training error) and **cross-validation** (direct estimation of test error).

6.2.1 Information Criteria

Information criteria adjust training RSS upward by a penalty that grows with model complexity, approximating the expected test error:

Criterion	Formula	Penalty	Choose model with
C_p (Mallows)	$\frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2)$	$2d\hat{\sigma}^2$	Smallest C_p
AIC	$-2\ell + 2d$	$2d$	Smallest AIC
BIC	$\frac{1}{n}(\text{RSS} + \log(n) \cdot d \cdot \hat{\sigma}^2)$	$\log(n) \cdot d \cdot \hat{\sigma}^2$	Smallest BIC
Adjusted R^2	$1 - \frac{\text{RSS}/(n-d-1)}{\text{TSS}/(n-1)}$	Implicit via $n-d-1$	Largest adj. R^2

where d is the number of predictors and $\hat{\sigma}^2$ is the estimated error variance from the full model.

AIC vs. BIC: Both penalise complexity, but BIC uses $\log(n)$ instead of 2 as the penalty multiplier. For $n \geq 8$, $\log(n) > 2$, so BIC penalises additional parameters more heavily than AIC and tends to select **sparser models**. For large n , BIC is consistent (selects the true model if it is among the candidates) while AIC tends to overfit slightly. In practice, BIC often gives more parsimonious models.

C_p and AIC are equivalent for OLS with Gaussian errors

Under Gaussian errors, maximising the log-likelihood ℓ is equivalent to minimising RSS (since $\ell = -n/2 \cdot \log(\text{RSS}/n) + \text{const}$). Therefore $\text{AIC} = -2\ell + 2d \propto \text{RSS}/\hat{\sigma}^2 + 2d \propto C_p$. The two criteria select the same model when used with OLS. They diverge for non-Gaussian models (logistic regression, etc.), where AIC is defined via the likelihood but C_p is not.

6.2.2 Cross-Validation for Model Selection

An alternative to information criteria is to directly estimate the test error of each model $\mathcal{M}_k$ using k -fold CV, then select the model with the lowest estimated test error. This approach:

- Makes no assumptions about the error distribution or the form of the model.
- Works for any type of model (not just OLS).
- Is slightly more expensive but often more reliable than information criteria when their assumptions are not met.

The **one-standard-error rule** is commonly applied: rather than picking the model with the absolute minimum CV error, choose the **simplest model** whose CV error is within one standard error of the minimum. This guards against overfitting to the variability in the CV estimate itself.

6.3 Shrinkage Methods: Ridge and Lasso

Shrinkage methods fit a model containing all p predictors but regularise the coefficient estimates by adding a **penalty term** to the loss function. The penalty shrinks coefficients toward zero, trading a small increase in bias for a potentially large reduction in variance.

6.3.1 Ridge Regression (L2 Regularisation)

Ridge regression minimises:

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \|\beta\|_2^2$$

The second term is the **L2 penalty** — the sum of squared coefficients. The tuning parameter $\lambda \geq 0$ controls the strength of regularisation:

- $\lambda = 0$: no penalty $\Rightarrow$ OLS solution.
- $\lambda \rightarrow \infty$: all $\hat{\beta}_j \rightarrow 0$ (except $\hat{\beta}_0 = \bar{y}$).
- Intermediate λ : coefficients are shrunk toward zero but not set exactly to zero.

Closed-form solution: Ridge has a closed-form solution (unlike Lasso):

$$\hat{\beta}_{\text{Ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

The addition of $\lambda \mathbf{I}$ makes $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})$ invertible even when $\mathbf{X}^\top \mathbf{X}$ is not — this is why Ridge works even when $p \geq n$. It also directly addresses multicollinearity: when two predictors are nearly collinear, OLS assigns very large and unstable coefficients of opposite sign; Ridge shrinks both toward zero, stabilising the estimates.

Note: The intercept β_0 is never penalised (it does not appear in the penalty sum). This is because penalising β_0 would make the predictions depend on the arbitrary choice of origin for Y . The intercept is estimated as $\hat{\beta}_0 = \bar{y}$ after centering.

Always standardise predictors before Ridge or Lasso

The L2 and L1 penalties are not scale-invariant: a predictor measured in kilometres has smaller coefficients than the same predictor measured in metres, and Ridge would shrink the latter more. Before applying Ridge or Lasso, **standardise all predictors** to have mean zero and unit variance:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\hat{\sigma}_{x_j}}$$

This ensures the penalty treats all predictors equally regardless of their measurement scale. The intercept is estimated separately and should not be standardised.

The bias-variance trade-off in Ridge: As λ increases, Ridge coefficients shrink, which: - **Reduces variance:** the model is less sensitive to the specific training data. - **Increases bias:** the model is constrained away from the true OLS solution.

The optimal λ minimises test MSE, which has the usual U-shape: too small λ gives high-variance OLS; too large λ gives an overly shrunk model with high bias. Choose λ by cross-validation.

6.3.2 The Lasso (L1 Regularisation)

The Lasso (Least Absolute Shrinkage and Selection Operator) replaces the L2 penalty with an **L1 penalty** — the sum of absolute values of coefficients:

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \|\beta\|_1$$

The critical difference from Ridge: The L1 penalty creates **corner solutions** — the Lasso can set some coefficients **exactly to zero**. This performs **automatic variable selection**: at sufficiently large λ , only a subset of predictors remains in the model. Ridge can only shrink coefficients toward zero, never to zero exactly.

Why corners arise: In the equivalent constrained formulation, Ridge constrains the solution to a sphere ($\sum \beta_j^2 \leq s$) and Lasso constrains it to a diamond ($\sum |\beta_j| \leq s$). The diamond has sharp corners on the coordinate axes. The OLS contours (ellipses) are likely to touch the diamond at a corner — where one or more $\beta_j = 0$ exactly — but much less likely to touch the sphere at a point on an axis.

No closed form: Unlike Ridge, Lasso has no closed-form solution because the L1 penalty is not differentiable at zero. It is solved numerically using **coordinate descent** or the LARS algorithm.

6.3.3 Ridge vs. Lasso: A Detailed Comparison

Property	Ridge (L2)	Lasso (L1)
Penalty term	$\lambda \sum_j \beta_j^2$	$\lambda \sum_j \beta_j $
Sets coefficients to exactly zero?	No — only shrinks toward zero	Yes — produces sparse solutions
Variable selection?	No — all predictors remain	Yes — built-in selection
Closed-form solution?	Yes: $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$	No — coordinate descent
With correlated predictors	Shrinks both together (similar coefficients)	Tends to pick one and zero out the rest

Property	Ridge (L2)	Lasso (L1)
Best when	Many predictors all have moderate effects	Few predictors have large effects; sparse truth
Works when $p > n$?	Yes	Yes (selects at most n predictors)
Interpretability	All predictors present; less interpretable	Sparse model; easier to interpret

Geometric intuition for why Lasso produces zeros but Ridge does not

Both Ridge and Lasso can be viewed as constrained minimisation problems:

- **Ridge**: minimise RSS subject to $\sum_j \beta_j^2 \leq s$ (a sphere / ball in $\mathbb{R}^p$)
- **Lasso**: minimise RSS subject to $\sum_j |\beta_j| \leq s$ (a diamond / polytope in $\mathbb{R}^p$)

The unconstrained OLS minimum lies outside the constraint region (otherwise no shrinkage would occur). The constrained solution is where the boundary of the constraint region first touches an elliptical contour of the RSS. The sphere has no corners — the touching point is almost surely an interior boundary point where no $\beta_j = 0$. The diamond *has* corners on the coordinate axes (where $\beta_j = 0$ for one or more j). The elliptical RSS contours are likely to first touch the diamond at a corner, giving a sparse solution.

6.3.4 Selecting λ by Cross-Validation

λ is a **tuning parameter** and must not be estimated on the training data (that would re-introduce the overfitting problem we are trying to solve). The standard procedure:

1. Define a grid of λ values, e.g. $\lambda \in \{10^{-4}, 10^{-3}, \dots, 10^4\}$ (logarithmically spaced)
2. For each λ in the grid, compute the k -fold CV error (using only training data)
3. Select $\lambda_{\min}$: the value with the lowest CV error
4. Optionally apply the **one-standard-error rule**: select λ_{1se} , the largest λ whose CV error is within one SE of the minimum. This gives a sparser (simpler) model with comparable performance
5. Refit the final model on the **full training set** using the selected λ

The **coefficient path** (plot of $\hat{\beta}_j(\lambda)$ vs. $\log \lambda$) is a useful visualisation. For Ridge, all coefficients shrink gradually to zero as λ increases. For Lasso, some coefficients reach zero at certain λ values (kinks in the path) and remain zero for larger λ .

6.4 Dimension Reduction: Principal Components Regression (PCR)

Rather than selecting or shrinking predictors, **dimension reduction** methods transform the p predictors into $M < p$ new variables (linear combinations) and regress Y on these M derived features.

Principal Components Regression (PCR) proceeds in two steps:

1. Compute the first M principal components $Z_1, \dots, Z_M$ of $\mathbf{X}$ (see Chapter 10 for full details on PCA). Each $Z_m = \sum_{j=1}^p \phi_{jm} X_j$ is a linear combination of the original predictors, chosen to explain the maximum variance in $\mathbf{X}$.
2. Regress Y on $Z_1, \dots, Z_M$ using OLS.

Key assumption: the directions of largest variation in $\mathbf{X}$ are also the directions most associated with Y . When this holds, using M principal components instead of all p predictors reduces variance (fewer effective parameters) at little cost in bias.

Choosing M : via cross-validation. $M = p$ recovers OLS; $M = 1$ gives the most reduced model.

PCR vs. Ridge: Ridge shrinks the coefficients of all p directions in $\mathbf{X}$ but more aggressively shrinks the directions of small variance. PCR discards the directions of small variance entirely and keeps the others unchanged. Both reduce effective dimensionality, but in different ways.

PCR is not the same as PCA

PCA (Principal Component Analysis) is an *unsupervised* method: it finds the directions of maximum variance in $\mathbf{X}$ without any reference to Y .

PCR uses these unsupervised components as predictors in a supervised regression. The components are chosen to explain $\mathbf{X}$, not to predict Y — this is a weakness. A direction that accounts for little variance in $\mathbf{X}$ might still be strongly associated with Y , and PCR would discard it.

Partial Least Squares (PLS) addresses this by finding components that maximise both the variance in $\mathbf{X}$ and the correlation with Y , but it is not covered in detail in this course.

Theory Questions – Chapter 6

Q1: Why does RSS always decrease (or stay flat) as we add more predictors to a linear model? Why can we not use RSS or R^2 directly to select model size, and what do we use instead?

A: When we add a predictor to a model, OLS can always set its coefficient to zero if it provides no value — in which case RSS stays exactly the same. If the predictor has any correlation with the response (even by chance in the training data), OLS will find a non-zero coefficient that reduces RSS. Therefore, RSS is non-increasing as predictors are added and $R^2 = 1 - \text{RSS}/\text{TSS}$ is non-decreasing. Both measures always favour the largest model, even if the added predictors are pure noise. To select model size, we need criteria that balance fit against complexity. The four standard approaches are: (1) C_p , which adds a penalty of $2d\hat{\sigma}^2$ to RSS; (2) AIC, proportional to C_p for Gaussian OLS; (3) BIC, with a heavier $\log(n) \cdot d\hat{\sigma}^2$ penalty that tends to select sparser models; and (4) adjusted R^2 , which penalises additional predictors through the degrees-of-freedom adjustment. Alternatively, k -fold CV directly estimates test error without any distributional assumptions.

Q2: Explain why Ridge regression can never set a coefficient exactly to zero, while Lasso can. What practical implication does this have?

A: The key is the geometry of the constraint region. Ridge constrains the solution to lie within an L2 ball (sphere): $\sum_j \beta_j^2 \leq s$. A sphere has no corners — its surface is perfectly smooth everywhere. The OLS contours (ellipses in 2D) will almost always touch the sphere at an interior boundary point, where no β_j equals zero. The Lasso constrains the solution to an L1 ball (diamond): $\sum_j |\beta_j| \leq s$. A diamond has sharp corners located exactly on the coordinate axes, where one or more $\beta_j = 0$. The OLS contours are much more likely to first contact the diamond at one of these corners than at an interior boundary point, producing sparse solutions with exactly-zero coefficients. The practical implication is that Lasso performs automatic variable selection: at a given λ , only a subset of predictors has non-zero coefficients, giving a model that is easier to interpret and potentially more parsimonious. Ridge keeps all predictors in the model at any $\lambda > 0$.

Q3: You fit a Ridge regression and a Lasso regression to the same dataset with λ chosen by 10-fold CV. The Ridge model uses all 50 predictors; the Lasso selects 8. The test MSEs are similar. Which model do you prefer, and why?

A: If the test MSEs are comparable, the Lasso model with 8 predictors is strongly preferred in most contexts. First, **interpretability**: a model with 8 predictors is far easier to understand and communicate — stakeholders can reason about which variables drive the outcome. Second, **parsimony**: the Lasso result suggests that the true signal may be concentrated in a small subset of predictors and the remaining 42 are largely noise. A simpler model is also less likely to overfit on future datasets with slightly different characteristics. Third, **variable selection**: in many applications (biology, economics, policy), identifying which predictors are relevant is itself valuable. The only scenario where Ridge might be preferred is if we believe all 50 predictors genuinely contribute (e.g. genomic data where many genes have small but real effects), in which case Lasso's tendency to arbitrarily pick one correlated predictor and drop others becomes a disadvantage.

7 Chapter 7 – Moving Beyond Linearity

The linear model is a powerful baseline, but it rests on the assumption that the true relationship between each predictor and the response is linear. When this assumption fails badly, predictions can be systematically wrong across entire regions of the input space. Chapter 7 introduces a family of methods that relax linearity while preserving the OLS fitting framework: polynomial regression, step functions, splines, local regression, and generalised additive models. A unifying idea connects almost all of them: the **basis function** framework.

7.1 Polynomial Regression

The simplest extension of linear regression replaces the single linear term $\beta_1 X$ with a polynomial of degree d :

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i$$

This is still a **linear model** — linear in the parameters $\beta_0, \dots, \beta_d$ — so the OLS estimator $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ applies directly, with a design matrix whose columns are $[1, x, x^2, \dots, x^d]$.

Degree selection: choose d via cross-validation. In practice, $d \leq 4$ is almost always sufficient: higher-degree polynomials become oscillatory and numerically unstable, especially near the boundaries of the data range (Runge’s phenomenon).

Limitations of polynomial regression:

- A single polynomial is a **global** function — a change in the fit at one point forces changes everywhere. There is no mechanism to fit data flexibly in one region while remaining smooth in another.
- Polynomials are especially unreliable at the **boundaries** (below the smallest x and above the largest x), where they can diverge wildly.
- For these reasons, regression splines are generally preferred over high-degree polynomials.

7.2 Step Functions

Step functions partition the range of X into $K + 1$ disjoint intervals using K **cutpoints** $c_1 < c_2 < \cdots < c_K$ and fit a separate constant within each interval. We create K indicator variables:

$$C_0(X) = \mathbf{1}(X < c_1), \quad C_1(X) = \mathbf{1}(c_1 \leq X < c_2), \quad \dots, \quad C_K(X) = \mathbf{1}(X \geq c_K)$$

Since $C_0 + C_1 + \cdots + C_K = 1$ always, we drop C_0 (to avoid the dummy variable trap) and fit:

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \cdots + \beta_K C_K(x_i) + \varepsilon_i$$

Here $\hat{\beta}_0$ is the mean response in the baseline bin C_0 , and $\hat{\beta}_k$ is the mean difference relative to the baseline in bin C_k .

Strength: the fit is constant within each bin — very interpretable (e.g. “average wage for age 20–30 vs. age 30–40”).

Weakness: step functions produce discontinuous fits — artificial jumps at the cutpoints. They also require the analyst to choose cutpoint locations, which can have a large influence on results. If the true relationship is smooth, step functions force a poor approximation.

7.3 Basis Functions: A Unifying Framework

Both polynomial regression and step functions are special cases of the **basis function** framework. We define K known functions $b_1(x), b_2(x), \dots, b_K(x)$ (the “basis”) and fit:

$$y_i = \beta_0 + \sum_{k=1}^K \beta_k b_k(x_i) + \varepsilon_i$$

This is still a linear model in the parameters $\beta_0, \dots, \beta_K$, so OLS applies with design matrix $[1, b_1(x), b_2(x), \dots, b_K(x)]$. The choice of basis functions determines the shape of the fit:

- $b_k(x) = x^k$: polynomial regression of degree K

- $b_k(x) = \mathbf{1}(c_{k-1} \leq x < c_k)$: step function
- $b_k(x) = (x - \xi_k)_+^3$: truncated power basis for cubic splines (see below)

The power of this framework is that it extends immediately to multiple predictors, classification (via GLMs), and other response types — always by choosing appropriate basis functions and fitting by OLS or MLE.

7.4 Regression Splines

Regression splines combine the local flexibility of step functions with the smoothness of polynomials by fitting piecewise polynomials subject to smoothness constraints at the join points (**knots**).

7.4.1 Piecewise Polynomials

A **piecewise cubic polynomial** with one knot at c fits two separate cubic polynomials:

$$\hat{f}(x) = \begin{cases} \beta_{01} + \beta_{11}x + \beta_{21}x^2 + \beta_{31}x^3 & \text{if } x < c \\ \beta_{02} + \beta_{12}x + \beta_{22}x^2 + \beta_{32}x^3 & \text{if } x \geq c \end{cases}$$

Without constraints, this has 8 free parameters. Each additional knot adds 4 parameters (one new cubic on the right side of the knot).

Problem: unconstrained piecewise polynomials can be discontinuous and have sharp bends at the knots. Adding continuity constraints makes the fit smoother.

7.4.2 Cubic Splines

A **cubic spline** with K knots at $\xi_1 < \xi_2 < \dots < \xi_K$ is a piecewise cubic polynomial that is constrained to be continuous and have continuous first and second derivatives at every knot. These $3K$ constraints reduce the effective degrees of freedom from $4(K+1)$ (unconstrained) to $K+4$:

$$\text{Degrees of freedom} = K + 4$$

This means a cubic spline with K knots uses $K+4$ basis functions. A convenient basis is the **truncated power basis**: start with the cubic polynomial basis $\{1, x, x^2, x^3\}$ (4 functions) and add one truncated power function per knot:

$$(x - \xi_k)_+^3 = \begin{cases} (x - \xi_k)^3 & \text{if } x > \xi_k \\ 0 & \text{otherwise} \end{cases}$$

Adding $(x - \xi_k)_+^3$ to the model introduces a cubic term that “turns on” to the right of knot ξ_k and is zero to the left. This exactly enforces continuity of f , f' , and f'' at ξ_k while allowing the cubic polynomial to change shape there.

Why $K+4$ degrees of freedom for a cubic spline?

Start with $K+1$ separate cubic polynomials: $(K+1) \times 4 = 4K+4$ parameters.

At each of the K knots, enforce 3 constraints (continuity of f , f' , f''): subtract $3K$ constraints.

Net degrees of freedom: $(4K+4) - 3K = K+4$.

Each additional knot adds exactly 1 effective degree of freedom (not 4), because 3 of the 4 new parameters are consumed by the smoothness constraints.

7.4.3 Natural Splines

A **natural cubic spline** (also called a natural spline) is a cubic spline with the additional constraint that the function is **linear** (not cubic) in the two boundary regions: $x < \xi_1$ and $x > \xi_K$. This removes 2 parameters from each end (the x^2 and x^3 terms), so:

$$\text{Degrees of freedom for natural spline} = K$$

The boundary linearity constraint reduces the degrees of freedom from $K + 4$ to K and provides much more **stable, less erratic estimates** near the edges of the data — the region where polynomials tend to blow up. In Python's `patsy`, natural splines are created with `cr(x, df=K)` (cubic regression splines).

Natural splines vs. cubic splines: Natural splines trade a slight increase in bias (forced linearity at the boundaries) for a substantial reduction in variance at the tails. Since we typically have less data near the boundaries, this trade-off is almost always beneficial.

7.4.4 Choosing the Number and Location of Knots

Two practical strategies:

1. **Place knots at quantiles of X :** if we want K knots, place them at the $100/(K + 1)$ th, $200/(K + 1)$ th, ..., $100K/(K + 1)$ th percentiles of the observed x values. This puts more knots where data are dense and fewer where data are sparse, adapting the spline's flexibility to the data distribution.
2. **Choose K by cross-validation:** compute the CV error for a range of knot counts ($K = 1, 2, \dots$) and select the value that minimises test error. The optimal K balances fit and flexibility.

Splines vs. high-degree polynomials: A cubic spline with K knots ($\text{df} = K + 4$) typically outperforms a polynomial of degree $K + 4$ because splines are **locally flexible** (the fit can change shape near each knot without affecting distant regions), while polynomials impose a single global shape. Splines are also more numerically stable.

7.5 Smoothing Splines

Smoothing splines take a different approach: rather than specifying knots in advance, find the function $g(x)$ that minimises the **penalised RSS**:

$$\text{Minimise over all smooth } g : \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int_{-\infty}^{\infty} [g''(t)]^2 dt$$

- The first term is the usual RSS: penalises lack of fit.
- The second term is the **roughness penalty**: $g''(t)$ is the second derivative (curvature) of g at t . Integrating the squared curvature over the entire domain penalises functions that wiggle a lot. $\lambda \geq 0$ controls the trade-off.

Behaviour as λ varies:

- $\lambda = 0$: no penalty — g can be any function that interpolates all n data points exactly ($g(x_i) = y_i$ for all i). Training RSS = 0, but the function may be extremely wiggly and overfit badly.
- $\lambda \rightarrow \infty$: the roughness penalty dominates — $g''(t) = 0$ everywhere, so g must be linear (a straight line, i.e. OLS).
- Intermediate λ : a smooth curve that trades off fit and roughness.

Key theoretical result: The function g that minimises the penalised criterion above is always a **natural cubic spline** with knots at every unique x value. The penalty λ effectively reduces the number of degrees of freedom the spline can use — even though there are n knots, the effective degrees of freedom $\text{df}_\lambda \in [2, n]$ (2 for a straight line, n for full interpolation).

Choosing λ : by **LOOCV**. A computational shortcut (analogous to the LOOCV shortcut for OLS from Chapter 5) makes this nearly as cheap as fitting the model once. In `statsmodels`, smoothing splines are accessed via `lowess` (approximate) or `UnivariateSpline`.

The effective degrees of freedom df_λ

The smoothing spline produces fitted values $\hat{\mathbf{g}} = \mathbf{S}_\lambda \mathbf{y}$, where $\mathbf{S}_\lambda$ is the $n \times n$ smoother matrix (analogous to the hat matrix $\mathbf{H}$ in OLS). The effective degrees of freedom are defined as:

$$\text{df}_\lambda = \text{tr}(\mathbf{S}_\lambda)$$

When $\lambda \rightarrow 0$: $\mathbf{S}_\lambda \rightarrow \mathbf{I}$ and $\text{df}_\lambda \rightarrow n$ (interpolation, full flexibility).

When $\lambda \rightarrow \infty$: $\mathbf{S}_\lambda \rightarrow \mathbf{H}_{\text{OLS}}$ for a linear fit and $\text{df}_\lambda \rightarrow 2$ (a straight line has 2 parameters).

Practitioners often specify df_λ directly (e.g. $\text{df}_\lambda = 10$) and let the software find the corresponding λ .

7.6 Local Regression

Local regression (also known as **LOESS** or **LOWESS** — Locally Weighted Scatterplot Smoothing) is a non-parametric smoother that fits a separate weighted regression at each prediction point x_0 , using only the observations nearest to x_0 .

1. Identify the k training observations closest to x_0 (by distance $|x_i - x_0|$). The fraction $s = k/n$ is called the **span**.
2. Assign observation-specific weights $w_i = K(x_i, x_0)$ based on distance: observations close to x_0 receive weight near 1; distant ones receive near 0. The **tricube kernel** is a common choice:

$$w_i = \left(1 - \left(\frac{|x_i - x_0|}{\max_j |x_j - x_0|} \right)^3 \right)^3$$

3. Fit a **weighted least squares** regression of y on x using only the k local observations with weights w_i . This is equivalent to minimising $\sum_{i \in \mathcal{N}_0} w_i (y_i - \beta_0 - \beta_1 x_i)^2$.
4. $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$ is the fitted value at x_0 from this local regression.
5. Repeat for every x_0 in the grid of prediction points.

The **span** s controls the bias-variance trade-off:

- **Small** s (e.g. $s = 0.2$): very local fit — only 20% of observations used at each point. Low bias (the local model closely follows the data), high variance (sensitive to individual observations), wiggly curve.
- **Large** s (e.g. $s = 0.8$): more global fit — 80% of observations used. High bias (forced to smooth over local features), low variance, smooth curve.

Choose s by cross-validation (LOOCV is common for local regression).

Local regression vs. KNN regression (Chapter 3): KNN regression also uses neighbours, but fits a simple mean. Local regression fits a weighted polynomial in each neighbourhood, which reduces **boundary bias** — the tendency of KNN to under- or over-shoot near the edges of the data range because neighbours are only available on one side.

Local regression in multiple dimensions ($p > 1$): LOWESS generalises to $p > 1$ predictors by computing multivariate distances. However, it suffers from the curse of dimensionality for $p > 3$ or 4: neighbourhoods must be large to contain enough observations, destroying the “local” advantage. GAMs are preferred in high dimensions.

7.7 Generalised Additive Models (GAMs)

GAMs extend multiple linear regression by replacing each linear term $\beta_j X_j$ with a **smooth function** $f_j(X_j)$, while maintaining **additivity**:

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon$$

Each f_j can be independently specified as a spline, smoothing spline, polynomial, local regression, or even a plain linear term $\beta_j X_j$. The model is additive — the total effect of the predictors is the sum of individual effects — but each effect can be arbitrarily non-linear.

7.7.1 Fitting GAMs: Backfitting

GAMs for regression are fitted using the **backfitting algorithm**, which iterates over predictors one at a time:

1. Initialise: $\hat{f}_j \equiv 0$ for all j ; $\hat{\beta}_0 = \bar{y}$
2. Cycle through $j = 1, 2, \dots, p$ (repeat until convergence):
 - (a) Compute the **partial residuals**: $r_i = y_i - \hat{\beta}_0 - \sum_{k \neq j} \hat{f}_k(x_{ik})$ (the residuals after removing the contributions of all other functions)
 - (b) Fit a smoother (e.g. smoothing spline, LOWESS) of r_i on x_{ij} to update $\hat{f}_j$

3. Repeat until the functions $\hat{f}_j$ stop changing (convergence is guaranteed for additive models)

7.7.2 GAMs for Classification (Logistic GAMs)

For a binary response, replace the linear predictor in logistic regression with additive smooth functions:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$$

Fitted using local scoring (iterated IRLS), available in Python via `pygam.LogisticGAM`.

7.7.3 Advantages and Disadvantages of GAMs

Aspect	Details
Flexibility	Each f_j can capture non-linear effects; more flexible than linear regression
Interpretability	Additive structure: plot each f_j separately to understand its effect on Y , holding others fixed
Mixed terms	Can freely mix smooth terms $f_j(X_j)$ and linear terms $\beta_j X_j$
Automatic smoothness	λ_j controls smoothness of each f_j , chosen by CV or GCV
Limitation — interactions	The additivity assumption means GAMs cannot model interactions between predictors. The effect of X_1 on Y cannot depend on X_2 . For interactions, add explicit interaction terms or use tree-based methods
Limitation — high-order interactions	With many predictors and complex interactions, trees and boosting often outperform GAMs

When to prefer each non-linear method

- **Polynomial regression:** simple, interpretable, works well when the non-linearity is mild and global. Avoid degree > 4 .
- **Regression splines / Natural splines:** flexible and smooth; more stable than high-degree polynomials; good when the non-linearity varies across the range of X . Preferred over polynomial regression in most cases.
- **Smoothing splines:** fully data-driven; no knot placement decision; λ controls smoothness. Use when you want maximum flexibility with automatic regularisation.
- **Local regression (LOWESS):** powerful for exploratory analysis and visualising trends; robust to outliers with appropriate kernel. Best for $p = 1$ or $p = 2$.
- **GAMs:** the natural extension to multiple predictors; interpretable; ideal when each predictor has an independent non-linear effect but interactions are not the primary concern.
- **Tree-based methods (Chapter 8):** preferred when strong interactions among predictors are expected, since trees automatically capture interactions through splits on multiple variables.

Theory Questions – Chapter 7

Q1: What is a natural cubic spline, and why is it preferred over a standard cubic spline at the boundaries of the data?

A: A natural cubic spline is a cubic spline with the additional constraint that the function is linear (i.e. the second and third derivative terms are zero) in the two boundary regions: below the smallest knot and above the largest knot. A standard cubic spline with K knots has $K + 4$ degrees of freedom; imposing linearity at each boundary removes 2 parameters at each end, reducing the degrees of freedom to K . The motivation for this constraint is that polynomials behave erratically near the boundaries of the data — there are few observations to anchor the fit, so cubic terms tend to diverge. By forcing linearity in the tails, natural splines produce more stable, better-behaved estimates in the boundary regions, at the cost of slightly increased bias there. Since the boundaries are precisely where data are sparse, this trade-off almost always benefits generalisation.

Q2: A smoothing spline is fitted with $\lambda = 0$ and separately with $\lambda \rightarrow \infty$. Describe what each fitted function looks like and what this implies about the role of λ .

A: The smoothing spline minimises $\sum_i (y_i - g(x_i))^2 + \lambda \int [g''(t)]^2 dt$. When $\lambda = 0$ there is no penalty on roughness, so the minimiser simply interpolates all data points: $g(x_i) = y_i$ for all i . The resulting function passes through every observation exactly (training RSS = 0) and may be extremely wiggly and overfit. When $\lambda \rightarrow \infty$ the roughness penalty dominates, forcing $g''(t) = 0$ everywhere — which means g must be a linear function (a straight line). This is equivalent to OLS with $p = 1$. For intermediate λ , the smoothing spline produces a smooth curve that balances fit (small RSS) against roughness (small $\int [g'']^2$). The parameter λ thus controls the effective degrees of freedom: from n (full interpolation, $\lambda = 0$) down to 2 (straight line, $\lambda \rightarrow \infty$). In practice, λ is chosen by LOOCV.

Q3: Explain how GAMs extend multiple linear regression to non-linear relationships. What is the key limitation of GAMs compared to tree-based methods?

A: A GAM replaces the linear terms $\beta_j X_j$ in multiple regression with arbitrary smooth functions $f_j(X_j)$, giving $Y = \beta_0 + \sum_j f_j(X_j) + \varepsilon$. Each f_j can be a spline, LOWESS, polynomial, or linear term, independently chosen for each predictor. The model is fitted by backfitting: iterating over predictors and fitting a univariate smoother to the partial residuals (residuals after removing all other f_k). This preserves additive structure while allowing each predictor to have a non-linear, data-driven relationship with Y . The key limitation is additivity: in a GAM, the effect of X_j on Y is the same regardless of the values of X_k for $k \neq j$ — interactions between predictors cannot be captured. For example, if the effect of age on income depends strongly on education level, a GAM would miss this interaction. Tree-based methods (random forests, gradient boosting) naturally capture such interactions through splits on multiple variables in sequence, at the cost of lower interpretability than the additive structure of GAMs.

8 Chapter 8 – Tree-Based Methods

Tree-based methods are among the most widely used methods in applied machine learning. They partition the predictor space into rectangular regions and make predictions using the mean (regression) or majority class (classification) within each region. A single decision tree is simple and interpretable, but tends to have high variance and moderate accuracy. Ensemble methods — bagging, random forests, and gradient boosting — address this by combining many trees, dramatically improving predictive performance at the cost of interpretability.

8.1 Regression Trees

8.1.1 Building a Regression Tree: Recursive Binary Splitting

A regression tree partitions the p -dimensional predictor space into J non-overlapping rectangular regions $R_1, R_2, \dots, R_J$. Within each region R_m , the prediction is the mean of the training responses that fall in that region:

$$\hat{y}_{R_m} = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} y_i$$

The goal is to find regions that minimise the total RSS:

$$\sum_{m=1}^J \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2$$

Finding the globally optimal partition is computationally infeasible, so we use **recursive binary splitting** — a top-down, greedy algorithm:

1. Start with all n observations in a single region (the root node)
2. At each step, consider all predictors $j = 1, \dots, p$ and all possible split points s for each predictor. For predictor j and split point s , define two half-planes:

$$R_1(j, s) = \{x \mid x_j < s\} \quad \text{and} \quad R_2(j, s) = \{x \mid x_j \geq s\}$$

Choose the (j, s) pair that minimises:

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

3. Split the current region using the chosen (j, s) , creating two child nodes
4. Repeat steps 2–3 within each child node until a stopping criterion is met (e.g. fewer than 5 observations per leaf)

Key properties: greedy (optimal at each step, not globally); top-down (splits are not reconsidered); produces axis-aligned rectangular regions.

Why is the algorithm greedy? At each step, recursive binary splitting chooses the single best split without considering how that split affects future splits. A split that is locally optimal may not be part of the globally optimal tree. This is the fundamental weakness of a single tree.

8.1.2 Tree Pruning: Cost-Complexity Pruning

Growing a large tree until every leaf contains only one observation gives a training error of zero — but wildly overfits. The solution is **pruning**: grow a large tree T_0 first, then remove branches that add little predictive power.

Pruning based solely on training RSS leads to overly large trees (the smallest terminal node always reduces training RSS). Instead, we use **cost-complexity pruning** (also called weakest-link pruning), which penalises tree size:

$$\text{RSS}_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha|T|$$

where $|T|$ is the number of **leaf nodes** (terminal nodes) in the subtree T , and $\alpha \geq 0$ is the complexity parameter (analogous to λ in Ridge/Lasso).

- $\alpha = 0$: no penalty on size $\Rightarrow$ the full tree T_0 is optimal.
- α large: each additional leaf must reduce RSS by more than α to justify its inclusion $\Rightarrow$ smaller trees.

1. Use recursive binary splitting to grow a large tree T_0 (stopping only when nodes are very small)
 2. Apply cost-complexity pruning: for each value of α , find the subtree $T(\alpha)$ that minimises $\text{RSS}_\alpha(T)$. As α increases, the optimal subtree shrinks monotonically
 3. Use k -fold CV to choose α : for each α in a grid, compute the CV error. Select $\hat{\alpha}$ with the lowest CV error (or apply the one-SE rule)
 4. Return the subtree $T(\hat{\alpha})$ fitted on the full training set
- In Python (`sklearn`): the pruning parameter is `ccp_alpha`. Use `cost_complexity_pruning_path()` to obtain the sequence of α values, then select via `GridSearchCV`.

8.2 Classification Trees

Classification trees are built the same way as regression trees, but since Y is categorical, the split criterion and prediction rule change:

- **Prediction**: within each leaf, predict the **majority class** (the most common class among training observations in that region).
- **Split criterion**: instead of RSS, use an **impurity measure** that quantifies how mixed the classes are within a node.

Let $\hat{p}_{mk}$ denote the proportion of training observations in region R_m belonging to class k . Three impurity measures:

Measure	Formula	Properties
Classification error	$1 - \max_k \hat{p}_{mk}$	Simple; equals 0 only when a node is pure; not differentiable — not preferred for growing trees
Gini index	$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$	Measures total variance across classes; equals 0 when a node is pure (one $\hat{p}_{mk} = 1$); preferred for growing trees
Cross-entropy (deviance)	$D = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$	Numerically similar to Gini; also equals 0 for pure nodes; slightly more computation

Gini index in depth: For a binary outcome ($K = 2$) with $\hat{p}_{m1} = p$:

$$G = p(1 - p) + (1 - p) \cdot p = 2p(1 - p)$$

This is maximised at $p = 0.5$ (maximum impurity / perfect mixing) and minimised at $p = 0$ or $p = 1$ (pure node). A split is chosen to maximise the reduction in weighted Gini impurity across the two child nodes.

Classification error vs. Gini/entropy: Classification error is not sufficiently sensitive to changes in node purity during tree growing (small moves in $\hat{p}_{mk}$ that improve purity may not change the majority class). Gini and entropy are more sensitive to purity and therefore preferred for determining splits. Classification error is used for final tree evaluation (it is the most interpretable performance metric).

Why Gini index is preferred over classification error for growing trees

Consider a node with 400 observations: 200 from class A, 200 from class B (50%/50%). Two possible splits:

Split 1: Left child: 300 A, 100 B. Right child: 100 A, 100 B.

Split 2: Left child: 200 A, 400 B (from another region). Right child: 0 A, 0 B (pure).

Both splits have the same classification error (one misclassification per node), but Split 2 creates one pure node while Split 1 creates none. The Gini index correctly prefers Split 2 because it is more sensitive to the purity improvement. Classification error would be indifferent between the two.

8.3 Advantages and Disadvantages of Trees

Advantages	Disadvantages
Highly interpretable; easy to visualise and explain to non-experts	High variance: small changes in training data can produce very different trees
Naturally handles non-linearity and interactions between predictors	Generally lower predictive accuracy than linear models when the true relationship is approximately linear
Works with both quantitative and qualitative predictors without dummy coding	Greedy splitting is not globally optimal
Mirrors human decision-making	Unstable: sensitive to outliers and near-threshold observations
Robust to irrelevant predictors (they simply do not get selected for splits)	Requires pruning to avoid overfitting
No need to standardise predictors (distance is not used)	Cannot extrapolate beyond training data ranges

The high variance of single trees is their main practical limitation — it motivates all the ensemble methods below.

8.4 Bagging (Bootstrap Aggregating)

Bagging reduces variance by averaging many trees, each trained on a bootstrap sample of the data:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

where $\hat{f}^{*b}$ is a tree grown on the b -th bootstrap sample. For classification, the final prediction is the **majority vote** across the B trees.

Why does averaging reduce variance? If B i.i.d. estimators each have variance σ^2 , their average has variance σ^2/B . Bootstrap trees are not truly independent (they are trained on overlapping datasets), so the variance reduction is less dramatic, but still substantial. Crucially, because each tree is grown deep (unpruned), individual bias remains low — bagging reduces variance without increasing bias.

Key hyperparameter: B (number of trees). More trees always improve or maintain performance — there is no overfitting as B increases. In practice, $B = 100$ – 500 is sufficient; returns diminish rapidly after a few hundred trees.

8.4.1 Out-of-Bag (OOB) Error

Recall from Chapter 5 that each bootstrap sample leaves out approximately 36.8% of observations. These **out-of-bag (OOB)** observations can be used to estimate test error without any additional CV:

1. For each observation i , identify all trees b for which i was **not** in the bootstrap sample (approximately $B/3$ trees)
2. Predict observation i using only these OOB trees: $\hat{y}_i^{\text{OOB}} = \frac{1}{|\{b: i \notin Z^{*b}\}|} \sum_{b: i \notin Z^{*b}} \hat{f}^{*b}(x_i)$
3. The OOB error is the MSE (regression) or misclassification rate (classification) of these OOB

predictions across all n observations
 OOB error is a valid, nearly unbiased estimate of test error and is approximately equivalent to LOOCV.
 It comes for free — no additional model fitting is needed.

8.4.2 Variable Importance from Bagged Trees

One downside of bagging is that averaging many trees destroys interpretability — we can no longer visualise a single decision tree. However, we can measure **variable importance**: for each predictor j , sum the total reduction in RSS (regression) or Gini impurity (classification) from all splits on predictor j , averaged over all B trees. A large total reduction indicates an important predictor.

$$\text{Importance}(X_j) = \frac{1}{B} \sum_{b=1}^B \sum_{\text{splits on } X_j \text{ in tree } b} \Delta \text{RSS}$$

Variable importance plots (bar charts sorted by importance) are the primary tool for understanding which predictors drive the ensemble's predictions.

8.5 Random Forests

Random forests improve on bagging by **decorrelating** the trees. In bagging, all B trees use the same set of p predictors — if there is one dominant predictor, virtually every tree will split on it near the root, making all trees highly correlated. Averaging many correlated estimates reduces variance far less than averaging independent ones.

The solution: at each split in each tree, consider only a **random subset of m predictors** (not all p):

$$\text{Default: } m = \sqrt{p} \text{ (classification), } m = p/3 \text{ (regression)}$$

By restricting the predictors at each split, trees are forced to use different predictors in different regions, making them less correlated. Averaging decorrelated trees reduces variance more effectively than averaging correlated bagged trees.

Bagging is a special case of Random Forests

When $m = p$ (all predictors available at each split), Random Forests reduces to exactly Bagging. As m decreases:

- Trees become more diverse (less correlated) $\rightarrow$ lower variance when averaged
 - Individual trees become weaker (they cannot always use the best predictor) $\rightarrow$ higher bias per tree
- The bias increase from using $m < p$ is typically more than offset by the variance reduction from decorrelation, so Random Forests with $m = \sqrt{p}$ almost always outperforms Bagging ($m = p$). The optimal m can be tuned by CV.

Key hyperparameters:

Parameter	Typical value	Effect
B (n_estimators)	100–500	More trees always helps (or neutral); no overfitting
m (max_features)	$\sqrt{p}$ (classification), $p/3$ (regression)	Lower $m \rightarrow$ more decorrelated trees, lower variance, but weaker individual trees
Max depth / min samples leaf	Unrestricted (grow deep)	Deep trees have low bias; variance controlled by averaging

Variable importance is computed identically to bagging — averaged over all B trees in the forest.

8.6 Gradient Boosting

Boosting takes a fundamentally different approach from bagging. Instead of fitting trees independently on bootstrap samples and averaging, boosting fits trees **sequentially**, each new tree correcting the errors of the current ensemble.

8.6.1 The Boosting Idea

The core intuition: fit a tree, look at where it was wrong (the residuals), then fit the next tree to those residuals. By repeatedly targeting the current mistakes, the ensemble gradually improves its fit to the training data.

1. Initialise: $\hat{f}(x) = 0$ and residuals $r_i = y_i$ for all i
2. For $b = 1, 2, \dots, B$:
 - (a) Fit a regression tree $\hat{f}^b$ with d splits (i.e. $d + 1$ leaves) to the data (x_i, r_i)
 - (b) Update the ensemble by adding a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \nu \hat{f}^b(x)$$

- (c) Update residuals: $r_i \leftarrow y_i - \hat{f}(x_i)$ (i.e. $r_i \leftarrow r_i - \nu \hat{f}^b(x_i)$)
3. Output: $\hat{f}(x) = \sum_{b=1}^B \nu \hat{f}^b(x)$

8.6.2 The Three Hyperparameters of Boosting

Boosting has three tuning parameters, all of which affect the bias-variance trade-off:

Hyperparameter	Symbol	Typical value	Role
Number of trees	B	100–5000 (chosen by CV)	More trees $\rightarrow$ lower bias, but can overfit if B is too large
Learning rate (shrinkage)	ν	0.001–0.1	Scales the contribution of each tree. Smaller ν requires larger B but generally gives better generalisation
Tree depth	d	1–6	Controls the complexity of each tree and the interaction depth. $d = 1$ (stumps) gives an additive model; $d = 2$ allows pairwise interactions; etc.

Interaction between ν and B : Smaller learning rate ν means each tree contributes less, so more trees B are needed to achieve the same fit. The combination of small ν and large B (e.g. $\nu = 0.01$, $B = 5000$) tends to give better generalisation than large ν and small B . In practice, set ν small and choose B by CV or early stopping on a validation set.

Tree depth d and interactions: Each split in a tree captures the effect of one variable conditionally on preceding splits. A tree of depth d can capture interactions among up to d variables. Stumps ($d = 1$) make boosting equivalent to an additive model (similar to a GAM). For most tabular data, $d = 2-4$ is sufficient.

Boosting can overfit — unlike bagging and random forests

In bagging and random forests, increasing B never hurts (the variance of the average can only decrease). In boosting, increasing B always reduces training error, but beyond a point it starts to overfit (memorise noise). Always choose B by cross-validation or monitor a held-out validation error during training and stop early when it stops improving.

8.6.3 Why Does Boosting Work? The Bias Perspective

Bagging and random forests primarily reduce **variance** — individual deep trees have low bias, and averaging reduces variance. Boosting primarily reduces **bias** — individual shallow trees (stumps) have high bias, and the sequential fitting of residuals progressively reduces it. This is why boosting uses shallow trees: the ensemble corrects for the bias of each tree by adding more trees. The result is a powerful method that can reduce both bias and variance simultaneously, which is why gradient boosting (XGBoost, LightGBM, CatBoost) consistently ranks among the top performers on tabular data benchmarks.

8.7 Summary of Ensemble Methods

Method	Sampling	Trees	Depth	Key advantage	Key disadvantage
Single tree	None	1	Pruned	Interpretable	High variance, moderate accuracy
Bagging	Bootstrap rows	B independent	Deep (unpruned)	Reduces variance	Trees correlated if strong predictor exists
Random Forest	Bootstrap rows + random features	B independent	Deep (unpruned)	Decorrelates trees, further reduces variance	Less interpretable; m to tune
Gradient Boosting	No bootstrap (full data)	B sequential	Shallow ($d = 1-6$)	Reduces bias AND variance; very high accuracy	3 hyperparameters to tune; can overfit

Choosing between Random Forests and Gradient Boosting

Random Forests are preferred when:

- Training time is limited (trees are fully independent; easy to parallelise)
- Robustness is important (gradient boosting is more sensitive to noisy labels and outliers)
- OOB error is needed (free test error estimate without CV)

Gradient Boosting is preferred when:

- Maximum predictive accuracy is the goal (boosting typically outperforms random forests when well-tuned)
- The data have a strong signal-to-noise ratio (less sensitive to overfitting when labels are clean)

In practice: try both, select by CV. For tabular data on Kaggle-style competitions, gradient boosting (XGBoost, LightGBM) dominates.

Theory Questions – Chapter 8

Q1: Explain the difference between the Gini index and classification error as split criteria for classification trees. Why is the Gini index preferred when growing trees?

A: Both measures quantify the impurity of a node — how mixed the class proportions are. Classification error is $1 - \max_k \hat{p}_{mk}$: it equals 0 when a node is pure and is maximised at $1 - 1/K$ when classes are equally distributed. The Gini index is $G = \sum_k \hat{p}_{mk}(1 - \hat{p}_{mk})$: it also equals 0 for pure nodes and is maximised at equal class proportions. The key difference is sensitivity: classification error only changes when the majority class changes, so small improvements in purity (e.g. a split that moves $\hat{p}$ from 0.51 to 0.70 for the majority class) are not reflected in the split criterion. The Gini index is sensitive to all changes in $\hat{p}_{mk}$, not just those that affect the majority class — it changes continuously as class

proportions change. This makes the Gini index a better guide for choosing splits during tree growing, since it consistently favours splits that create purer nodes. Classification error is still used for reporting final test performance because it has a direct, interpretable meaning (fraction of observations misclassified).

Q2: Why do random forests outperform bagging in most settings? What is the role of the hyperparameter m ?

A: Bagging reduces variance by averaging B independent trees trained on bootstrap samples. However, if one predictor is strongly associated with Y , nearly every bagged tree will split on it near the root, making the trees highly correlated. Averaging correlated random variables reduces variance by a factor of roughly $1/B \cdot (1 - \rho) + \rho$ (where ρ is the correlation), which is much less than $1/B$ when ρ is large. Random forests break this correlation by considering only a random subset of m predictors at each split. This forces different trees to use different predictors, producing more diverse (less correlated) trees. The trade-off is that individual trees are slightly weaker (they cannot always use the best predictor), but the variance reduction from decorrelation more than compensates, giving better generalisation. The hyperparameter m directly controls this trade-off: smaller m produces more diverse, less correlated trees (lower variance) but weaker individual trees (higher bias). The defaults $m = \sqrt{p}$ (classification) and $m = p/3$ (regression) work well in practice and can be tuned by CV.

Q3: Explain how gradient boosting differs from bagging in both its mechanism and the type of error it primarily reduces. Why can gradient boosting overfit while bagging cannot?

A: Bagging fits B trees **independently** on bootstrap samples of the training data and averages their predictions. Each tree is grown deep (low bias, high variance) and the averaging operation reduces variance. The key insight is that averaging does not change expected bias — it only reduces variance. Therefore, bagging primarily reduces variance. Because each additional tree is independent and its contribution is averaged into a larger ensemble, adding more trees can only help or be neutral — bagging cannot overfit. Gradient boosting fits trees **sequentially**: each tree is fitted to the current residuals (the errors of the existing ensemble). The learning rate ν scales each tree's contribution. Because each tree targets the mistakes of the previous ensemble, boosting progressively reduces bias — individual trees are shallow (high bias, low variance) and the sequential process reduces their collective bias. However, as more trees are added, the ensemble increasingly memorises the training data's noise (especially with large ν), which increases variance and causes overfitting. This is why B must be chosen carefully by cross-validation or early stopping, unlike in bagging where more trees are always at least as good.

9 Chapter 9 – Support Vector Machines

Support Vector Machines (SVMs) are a family of powerful classifiers built around the geometric concept of a separating hyperplane. The SVM idea develops progressively through three stages: (1) the maximal margin classifier for perfectly separable data, (2) the support vector classifier that tolerates some misclassification, and (3) the full SVM that uses the kernel trick to handle non-linear decision boundaries. SVMs are particularly effective in high-dimensional settings and remain competitive with ensemble methods when well-tuned.

9.1 What is a Hyperplane?

In p -dimensional space, a **hyperplane** is a flat affine subspace of dimension $p - 1$. It is defined by:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0$$

- In $p = 2$ dimensions, a hyperplane is a **line**.
- In $p = 3$ dimensions, a hyperplane is a **plane**.
- In $p > 3$ dimensions, it is a $(p - 1)$ -dimensional flat surface.

The hyperplane divides $\mathbb{R}^p$ into two half-spaces:

$$\begin{aligned} \beta_0 + \beta^\top \mathbf{x} &> 0 &\Leftrightarrow \mathbf{x} \text{ is on the positive side} \\ \beta_0 + \beta^\top \mathbf{x} &< 0 &\Leftrightarrow \mathbf{x} \text{ is on the negative side} \end{aligned}$$

The magnitude $|\beta_0 + \beta^\top \mathbf{x}|$ (with $\|\beta\| = 1$) is the **perpendicular distance** from point $\mathbf{x}$ to the hyperplane. This signed distance forms the basis for all SVM classifiers.

We encode class labels as $y_i \in \{-1, +1\}$ (rather than $\{0, 1\}$), which simplifies the mathematical notation: a classifier based on the hyperplane predicts $\hat{y}_i = \text{sign}(\beta_0 + \beta^\top \mathbf{x}_i)$.

9.2 The Maximal Margin Classifier (Hard Margin SVM)

When two classes are **linearly separable** (a hyperplane exists that perfectly separates all training observations), there are infinitely many separating hyperplanes. The **maximal margin classifier** selects the one that maximises the **margin** — the total perpendicular distance from the hyperplane to the nearest training observations on each side.

Formally, let the hyperplane be $\{\mathbf{x} : \beta_0 + \beta^\top \mathbf{x} = 0\}$ with $\|\beta\| = 1$. The margin is $2M$ where M is the distance from the hyperplane to the nearest point. We solve:

$$\max_{\beta_0, \beta, M} M \quad \text{subject to} \quad y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geq M \quad \forall i, \quad \|\beta\| = 1$$

The constraint $y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geq M$ ensures every observation is correctly classified and lies at least M units from the hyperplane (since $y_i \in \{-1, +1\}$, the product is positive when correctly classified).

9.2.1 Support Vectors

The observations that lie exactly on the margin boundaries — i.e. those for which $y_i(\beta_0 + \beta^\top \mathbf{x}_i) = M$ — are called **support vectors**. They are the only observations that determine the hyperplane. All other observations could be moved without changing the solution, as long as they remain on the correct side of their margin boundary.

Why "support vectors"?

These observations literally "support" the hyperplane: they are the points closest to the decision boundary and define (support) where the margin lies. The hyperplane is entirely determined by the support vectors; observations far from the boundary have no influence on it. This is the reason SVMs can be effective even in high-dimensional spaces — only a small number of observations matter.

9.2.2 Limitations of the Maximal Margin Classifier

- **Fails when data are not linearly separable:** no feasible solution exists when classes overlap.
- **Extremely sensitive to individual observations:** because the hyperplane is determined only by the support vectors, a single new observation near the boundary can completely change the hyperplane — very high variance.
- **Overfitting risk:** the hardest constraint (all observations must be correctly classified) may produce a very narrow margin that generalises poorly.

9.3 The Support Vector Classifier (Soft Margin SVM)

The support vector classifier (SVC) relaxes the hard margin constraint by allowing some observations to be on the wrong side of the margin or even the wrong side of the hyperplane. This is controlled via **slack variables** $\xi_i \geq 0$, one per observation:

$$\max_{\beta_0, \beta, M, \xi_i} M \quad \text{subject to: } y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geq M(1 - \xi_i), \quad \xi_i \geq 0, \quad \sum_{i=1}^n \xi_i \leq C, \quad \|\beta\| = 1$$

9.3.1 Interpreting the Slack Variables

Value of ξ_i	Location of observation i
$\xi_i = 0$	Correctly classified and on or beyond the margin boundary — no violation
$0 < \xi_i \leq 1$	On the correct side of the hyperplane but inside the margin (between margin and boundary)
$\xi_i > 1$	On the wrong side of the hyperplane — misclassified

The total slack $\sum_i \xi_i$ is bounded by C , which serves as a budget for violations.

9.3.2 The Role of C (Budget Parameter)

C is the most important tuning parameter of the SVC and directly controls the bias-variance trade-off:

- $C = 0$: no violations allowed $\Rightarrow$ reduces to the hard-margin maximal margin classifier (only valid if classes are separable).
- **Large C :** a large violation budget — the classifier tolerates many observations inside the margin or even misclassified. This produces a **wide margin** with many support vectors. The classifier is less sensitive to individual observations (lower variance, higher bias). In sklearn, this corresponds to a **small C** (sklearn parameterises by $1/C$, so large C in sklearn means strict margin).
- **Small C :** a tight violation budget — very few violations allowed. The margin is **narrow** with fewer support vectors. The classifier fits the training data more closely (higher variance, lower bias). In sklearn, large C means strict margin and small C means soft margin.

sklearn's C parameter is the inverse of the "budget"

In ISLP (and most textbooks), a **larger C** budget means **more** violations are allowed $\rightarrow$ **wider** margin $\rightarrow$ **lower** variance. In sklearn's SVC, the parameter C is the **regularisation parameter** (penalty on margin violations): **large C** $\rightarrow$ **few** violations tolerated $\rightarrow$ **narrow** margin $\rightarrow$ **higher** variance. The two conventions are opposite. On the exam, always clarify which convention you are using.

Observations that influence the classifier: only observations that either lie on the margin boundary or violate the margin (i.e. those with $\xi_i > 0$ — inside or on the wrong side) become support vectors and influence $\hat{f}$. Correctly classified observations far from the boundary have $\xi_i = 0$ and play no role.

Choose C by cross-validation.

9.4 The Support Vector Machine (Kernel Trick)

The SVC finds a linear decision boundary in the original feature space. For non-linearly separable data, we could manually add polynomial or interaction features — but this is computationally expensive in high dimensions. The **kernel trick** achieves the same effect implicitly and efficiently.

9.4.1 The Dual Formulation and Inner Products

A key mathematical insight: the SVC solution can be written entirely in terms of **inner products** between observations. The decision function takes the form:

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle$$

where $\alpha_i \geq 0$ are Lagrange multipliers. Crucially, $\alpha_i > 0$ only for the support vectors — all other observations have $\alpha_i = 0$ and do not contribute. The inner product $\langle \mathbf{x}_i, \mathbf{x} \rangle = \mathbf{x}_i^\top \mathbf{x}$ measures the similarity between training observation i and test point $\mathbf{x}$.

The kernel trick: replace every inner product $\langle \mathbf{x}_i, \mathbf{x} \rangle$ with a **kernel function** $K(\mathbf{x}_i, \mathbf{x})$:

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x})$$

where $\mathcal{S}$ is the set of support vectors. The kernel function computes a generalised similarity measure that implicitly corresponds to computing the inner product in a high-dimensional (possibly infinite-dimensional) feature space, without ever explicitly constructing that space. This makes it computationally tractable even when the expanded feature space would be enormous.

9.4.2 Common Kernel Functions

Kernel	Formula	Decision boundary	Key parameter
Linear	$K(\mathbf{x}_i, \mathbf{x}_{i'}) = \mathbf{x}_i^\top \mathbf{x}_{i'}$	Linear (standard SVC)	None
Polynomial (degree d)	$K(\mathbf{x}_i, \mathbf{x}_{i'}) = (1 + \mathbf{x}_i^\top \mathbf{x}_{i'})^d$	Polynomial of degree d	d
RBF / Gaussian	$K(\mathbf{x}_i, \mathbf{x}_{i'}) = \exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_{i'}\ ^2)$	Arbitrarily non-linear	γ

The **RBF (Radial Basis Function) kernel** is by far the most commonly used in practice. It gives the highest weight to training observations close to the test point (small $\|\mathbf{x}_i - \mathbf{x}_{i'}\|^2$) and near-zero weight to distant observations. It implicitly corresponds to an infinite-dimensional feature space and can represent arbitrarily complex decision boundaries.

Intuition for the RBF kernel

The RBF kernel $K(\mathbf{x}_i, \mathbf{x}_{i'}) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_{i'}\|^2)$ is essentially a Gaussian function of the squared Euclidean distance between two observations. When $\mathbf{x}_{i'} = \mathbf{x}_i$ (the same point), $K = 1$ (maximum similarity). As $\mathbf{x}_{i'}$ moves away from $\mathbf{x}_i$, K decreases toward 0. The parameter γ controls the rate of this decay:

- **Small γ :** the kernel decays slowly — distant observations still have meaningful similarity. The decision boundary is smooth and global (high bias, low variance).
- **Large γ :** the kernel decays rapidly — only very close observations are considered similar. The decision boundary becomes highly local and complex (low bias, high variance, risk of overfitting).

9.4.3 Choosing C and γ by Cross-Validation

Both C and γ (for the RBF kernel) must be chosen by cross-validation. A common approach is **grid search**: try all combinations of C and γ from a candidate grid and select the combination with the lowest CV error.

	Low value	High value
C (sklearn convention)	Wide margin; more violations; lower variance, higher bias	Narrow margin; fewer violations; higher variance, lower bias
γ (RBF kernel)	Smooth, global boundary; high bias, low variance	Complex, local boundary; low bias, high variance; risk of overfit

The optimal combination balances the flexibility of the boundary (controlled by γ) against the tolerance for misclassification (controlled by C). A common starting grid: $C \in \{0.01, 0.1, 1, 10, 100\}$, $\gamma \in \{0.001, 0.01, 0.1, 1, 10\}$ — using a logarithmic scale in both directions.

The key advantage of kernels: efficient high-dimensional computation

Suppose we want a polynomial boundary of degree $d = 3$ in $p = 100$ dimensions. Explicitly constructing all polynomial features would involve $\binom{100+3}{3} \approx 176,851$ features — storing and computing with these is expensive. The polynomial kernel $K(\mathbf{x}_i, \mathbf{x}_{i'}) = (1 + \mathbf{x}_i^\top \mathbf{x}_{i'})^3$ produces the same decision boundary as if we had expanded to the full degree-3 feature space, but requires only a single dot product plus a few arithmetic operations per pair of observations. The kernel trick is what makes SVMs tractable in high-dimensional settings.

9.5 Multi-Class SVM

The basic SVM is a binary classifier ($K = 2$ classes). For $K > 2$ classes, two strategies extend it:

9.5.1 One-vs-One (OvO)

Train $\binom{K}{2} = K(K-1)/2$ binary SVMs, one for each pair of classes. For each test observation, apply all $K(K-1)/2$ classifiers and assign the observation to the class that wins the most pairwise contests (majority vote). This is the **default in sklearn**.

Example: with $K = 4$ classes, we train $\binom{4}{2} = 6$ binary SVMs: (1 vs 2), (1 vs 3), (1 vs 4), (2 vs 3), (2 vs 4), (3 vs 4). Each binary classifier votes for one class; the class with 3 or more votes (out of 6) wins.

9.5.2 One-vs-All (OvA)

Train K binary SVMs, each comparing one class against all others combined. For test observation $\mathbf{x}$, compute the decision function $f_k(\mathbf{x})$ for each of the K SVMs and assign to the class with the highest score:

$$\hat{y} = \arg \max_k f_k(\mathbf{x})$$

OvA requires training only K classifiers (vs. $K(K-1)/2$ for OvO) and is therefore faster for large K , but class imbalance can be more severe (each classifier has n_k positives vs. $n - n_k$ negatives).

Strategy	Number of SVMs	Vote mechanism	Default in sklearn
One-vs-One (OvO)	$K(K-1)/2$	Majority vote across pairwise contests	Yes
One-vs-All (OvA)	K	Assign to class with highest $f_k(\mathbf{x})$	No (<code>decision_function_shape='ovr'</code>)

9.6 SVM vs. Other Classifiers

Aspect	SVM (RBF kernel)	Logistic Regression	Random Forest
Decision boundary	Arbitrary non-linear	Linear	Piecewise axis-aligned
Works well in high p	Yes (kernel trick; few support vectors)	Yes	Less so (curse of dimensionality for trees)
Interpretability	Low (no coefficients; only support vectors)	High (log-odds coefficients)	Moderate (variable importance)
Probabilistic output	Requires extra calibration (Platt scaling)	Direct probabilities	Direct probabilities
Key hyperparameters	C, γ (RBF)	Regularisation C or λ	B, m, depth
Best when	Clear margin of separation; moderate n , large p	Linear boundary; need interpretability	Large n ; complex interactions

Theory Questions – Chapter 9

Q1: What is the margin in the maximal margin classifier, and why do we want to maximise it? What are the support vectors?

A: The margin is the perpendicular distance between the decision hyperplane and the nearest training observations on each side. Maximising the margin produces the hyperplane that is furthest from all training observations — intuitively, a larger margin means the classifier is more "confident" about its predictions and more robust to small perturbations in the data or new observations close to the boundary. Observations that lie exactly on the margin boundaries — i.e. those with $y_i(\beta_0 + \beta^\top \mathbf{x}_i) = M$ — are the support vectors. They are the only observations that determine the position and orientation of the hyperplane; all other observations could be moved (as long as they remain on the correct side of the margin) without changing the solution. This makes the maximal margin classifier computationally efficient (only support vectors matter) but sensitive to individual boundary observations.

Q2: Explain the role of the slack variables ξ_i in the support vector classifier. What does it mean when $\xi_i = 0$, $0 < \xi_i \leq 1$, and $\xi_i > 1$?

A: The slack variables $\xi_i \geq 0$ allow the support vector classifier to tolerate observations that violate the margin or are misclassified. The constraint $y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geq M(1 - \xi_i)$ with $\sum_i \xi_i \leq C$ permits up to a total budget C of violation. When $\xi_i = 0$, observation i is correctly classified and lies on or beyond the margin boundary — no violation. When $0 < \xi_i \leq 1$, observation i is on the correct side of the hyperplane but has crossed into the margin: it is correctly classified but within the margin zone (between the margin boundary and the decision hyperplane). When $\xi_i > 1$, observation i is on the wrong side of the hyperplane and is misclassified. The budget parameter C controls the total amount of violation tolerated: large C allows many violations, producing a wider margin and a lower-variance (but higher-bias) classifier; small C allows few violations, producing a narrower margin and a higher-variance (but lower-bias) classifier.

Q3: What is the kernel trick, and why is it essential for SVMs to handle non-linear boundaries? Explain intuitively what the RBF kernel computes.

A: The SVC finds a linear boundary in the original feature space. To handle non-linear boundaries, we could expand the feature space by adding polynomial or interaction terms, but this becomes computationally prohibitive in high dimensions. The kernel trick exploits the fact that the SVC's decision function depends on the data only through pairwise inner products $\langle \mathbf{x}_i, \mathbf{x}_{i'} \rangle$. By replacing these inner products with a kernel function $K(\mathbf{x}_i, \mathbf{x}_{i'})$, we implicitly map the data into a higher-dimensional (possibly infinite-dimensional) feature space and compute inner products there, without ever explicitly constructing that space. This is computationally efficient because the kernel evaluation requires only a single function call per pair of observations. The RBF (Gaussian) kernel $K(\mathbf{x}_i, \mathbf{x}_{i'}) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_{i'}\|^2)$ measures similarity as a decreasing function of the Euclidean distance between two observations. It assigns high similarity (close to 1) to observations that are nearby and near-zero similarity to distant observations. With small γ , the similarity decays slowly and the boundary is smooth; with large γ , only very close observations are considered similar, producing a highly local and potentially very complex boundary. The SVM with RBF kernel corresponds to an infinite-dimensional feature space, making it capable of representing arbitrarily complex decision boundaries.

10 Chapter 10 – Deep Learning and Neural Networks

Neural networks are a family of flexible, non-linear models that can approximate virtually any function given enough hidden units and data. They underpin modern deep learning and achieve state-of-the-art performance on image recognition, language modelling, and many structured prediction tasks. For tabular data — the typical setting in BAN404 exams — neural networks compete with gradient boosting, and which is better depends on the dataset. Understanding the architecture, training procedure, and regularisation of neural networks is essential.

10.1 From Linear to Non-Linear: The Need for Activation Functions

A linear model computes $\hat{Y} = \mathbf{X}\beta$. A neural network with one hidden layer computes:

$$A_k = h\left(\beta_{0k} + \sum_{j=1}^p \beta_{jk} X_j\right), \quad k = 1, \dots, K \quad (\text{hidden layer})$$

$$\hat{Y} = g_0 + \sum_{k=1}^K g_k A_k \quad (\text{output layer})$$

If the activation function h were linear (i.e. $h(z) = z$), then the hidden layer would be a linear transformation of X and the output would be a linear function of a linear function — which is still a linear function of X . **Non-linear activation functions are essential:** without them, stacking multiple layers provides no benefit over a single linear model.

10.2 Architecture of a Multilayer Perceptron (MLP)

An MLP consists of an input layer, one or more hidden layers, and an output layer. Information flows forward from inputs to output (forward pass).

Layer	Description	Size
Input layer	One node per predictor; no computation — simply passes X_j to the first hidden layer	p nodes
Hidden layer l	Each node computes a weighted sum of the previous layer's outputs, then applies an activation function h	K_l nodes
Output layer	Produces the final prediction; activation function depends on the task	1 node (regression/binary); K nodes (K -class)

For a network with L hidden layers of sizes $K_1, K_2, \dots, K_L$:

$$\mathbf{h}^{(l)} = h\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \dots, L$$

$$\hat{Y} = g\left(\mathbf{W}^{(L+1)}\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}\right)$$

where $\mathbf{W}^{(l)}$ is the weight matrix and $\mathbf{b}^{(l)}$ is the bias vector for layer l , and h and g are the hidden and output activation functions respectively.

Total parameters: For a single hidden layer with K nodes, the total number of trainable parameters is:

$$\underbrace{p \cdot K}_{\text{input-to-hidden weights}} + \underbrace{K}_{\text{hidden biases}} + \underbrace{K}_{\text{hidden-to-output weights}} + \underbrace{1}_{\text{output bias}} = pK + 2K + 1$$

Each additional hidden layer multiplies the parameter count further. Neural networks are **overparameterised** models — they typically have far more parameters than observations, making regularisation essential.

Universal approximation theorem

A single hidden layer MLP with enough hidden units (K large) and a non-linear activation function can approximate any continuous function on a compact domain to arbitrary precision. This is the theoretical justification for using neural networks as general-purpose function approximators.

However, "enough hidden units" may be astronomically large for complex functions. In practice, **deep networks** (many layers with moderate width) learn complex functions more efficiently than a single wide hidden layer, because successive layers learn hierarchical representations (e.g. edges $\rightarrow$ shapes $\rightarrow$ objects in image recognition).

10.3 Activation Functions

The activation function h introduces non-linearity into the network. Different functions are used in hidden layers vs. output layers:

Function	Formula	Range	Typical use
ReLU	$h(z) = \max(0, z)$	$[0, \infty)$	Hidden layers — default choice; fast; avoids vanishing gradients
Sigmoid	$h(z) = \frac{1}{1 + e^{-z}}$	$(0, 1)$	Output layer for binary classification
Softmax	$h(z_k) = \frac{e^{z_k}}{\sum_{l=1}^K e^{z_l}}$	$(0, 1)$, sums to 1	Output layer for K -class classification
Tanh	$h(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Hidden layers; zero-centred version of sigmoid
Linear	$h(z) = z$	$(-\infty, \infty)$	Output layer for regression

10.3.1 Why ReLU is Preferred in Hidden Layers

- **Computationally efficient:** $\max(0, z)$ is trivially fast to evaluate and differentiate.
- **Avoids the vanishing gradient problem:** sigmoid and tanh saturate (flat derivatives) for large $|z|$, causing gradients to vanish during backpropagation and making deep networks very slow to train. ReLU has a gradient of exactly 1 for $z > 0$, preserving gradient flow.
- **Sparsity:** approximately half the ReLU units are zero at any given time, creating sparse activations that act as implicit regularisation.

Why softmax for multi-class output?

For a K -class classification problem, the output layer needs to produce K probabilities that sum to 1. The softmax function $h(z_k) = e^{z_k} / \sum_l e^{z_l}$ achieves this: it exponentiates all K logits $z_1, \dots, z_K$ and normalises so they sum to 1. The class with the highest logit gets the highest probability. This is the multi-class generalisation of the sigmoid: for $K = 2$, softmax reduces to sigmoid applied to the difference $z_1 - z_2$.

10.4 Loss Functions

The network is trained by minimising a **loss function** that measures how far predictions are from the truth. The appropriate loss depends on the prediction task:

Task	Output activation	Loss function	Formula
Regression	Linear	Mean Squared Error (MSE)	$\frac{1}{n} \sum_i (y_i - \hat{f}(x_i))^2$
Binary classification	Sigmoid	Binary cross-entropy	$-\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$
Multi-class classification	Softmax	Categorical cross-entropy	$-\frac{1}{n} \sum_i \sum_{k=1}^K y_{ik} \log \hat{p}_{ik}$

Binary cross-entropy is the negative log-likelihood of the Bernoulli distribution. Minimising it is equivalent to maximum likelihood estimation for logistic regression — so a single sigmoid-output unit with binary cross-entropy loss is mathematically equivalent to logistic regression with a highly non-linear feature space constructed by the hidden layers.

Categorical cross-entropy with softmax output is the multi-class generalisation of MLE for multinomial logistic regression.

10.5 Training: Backpropagation and Gradient Descent

Neural network training finds weights $\mathbf{W}^{(l)}$ and biases $\mathbf{b}^{(l)}$ that minimise the loss $\mathcal{L}$. Unlike OLS regression, there is **no closed-form solution** — the loss surface is non-convex with many local minima and saddle points. Iterative gradient-based optimisation is used.

10.5.1 The Gradient Descent Update Rule

The fundamental update rule: move the weights in the direction of steepest descent of the loss:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$$

where $\eta > 0$ is the **learning rate** — the step size per gradient update. Computing $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ requires the **backpropagation algorithm**.

10.5.2 Backpropagation

Backpropagation is an efficient application of the **chain rule** of calculus to compute gradients of the loss with respect to all weights in the network. It propagates the gradient backward from the output layer through the hidden layers:

Forward pass (compute predictions and loss):

1. Pass input $\mathbf{x}_i$ through each layer, computing activations $\mathbf{h}^{(l)}$ and finally $\hat{y}_i$
2. Compute the loss $\mathcal{L}(\hat{y}_i, y_i)$

Backward pass (compute gradients via chain rule):

1. Compute $\partial \mathcal{L} / \partial \hat{y}$ (gradient of loss w.r.t. output)
2. For each layer $l = L, L-1, \dots, 1$ (going backward):
 - (a) Compute $\partial \mathcal{L} / \partial \mathbf{z}^{(l)}$ using the chain rule and the gradient from layer $l+1$
 - (b) Compute $\partial \mathcal{L} / \partial \mathbf{W}^{(l)} = \partial \mathcal{L} / \partial \mathbf{z}^{(l)} \cdot (\mathbf{h}^{(l-1)})^\top$
3. Update all weights: $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \partial \mathcal{L} / \partial \mathbf{W}^{(l)}$

10.5.3 Mini-Batch Stochastic Gradient Descent

Computing the loss and gradient over all n observations at each step (full batch gradient descent) is expensive for large n . **Mini-batch SGD** approximates the gradient using a random subset (mini-batch) of $B_{\text{mb}} \approx 32\text{--}256$ observations per update:

- **Epoch**: one complete pass through the entire training dataset.
- **Iteration**: one gradient update using one mini-batch.
- One epoch = $\lceil n / B_{\text{mb}} \rceil$ iterations.

Mini-batch SGD introduces noise into the gradient estimates, which helps escape shallow local minima and saddle points. The noise also acts as an implicit regulariser.

10.5.4 Adaptive Optimisers

Plain SGD with a fixed learning rate often converges slowly or oscillates. **Adaptive optimisers** automatically adjust the learning rate for each parameter:

- **Adam** (Adaptive Moment Estimation): maintains a moving average of both the gradient and its squared magnitude, giving effective per-parameter learning rates. Strongly recommended as the default for neural networks.

- **RMSProp**: divides the learning rate by a running average of recent squared gradients, preventing the learning rate from becoming too small in directions with consistent gradients.

In practice: use **Adam** with a moderate initial learning rate (e.g. $\eta = 0.001$) unless there is a specific reason to prefer plain SGD.

10.6 Hyperparameters of Neural Networks

Neural networks have many hyperparameters. Unlike model parameters ($\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$), hyperparameters are not learnt from data — they must be set before training:

Hyperparameter	Typical range	Effect
Number of hidden layers L	1–5 (tabular); up to 100+ (images, text)	More layers $\rightarrow$ more complex feature hierarchies; harder to train; more regularisation needed
Nodes per layer K_l	16–1024	Wider layers $\rightarrow$ more parameters; higher capacity; higher variance
Learning rate η	10^{-4} – 10^{-2}	Too high $\rightarrow$ divergence or oscillation; too low $\rightarrow$ very slow convergence
Mini-batch size B_{mb}	32–256	Larger batches $\rightarrow$ more stable gradients; less noise; may need adjusted η
Number of epochs	10–1000	Train until validation loss stops improving (early stopping)
Dropout rate p_{drop}	0.2–0.5	See regularisation section
L2 weight decay λ	10^{-4} – 10^{-2}	See regularisation section

Tuning strategy: for tabular data, start with a moderate architecture (2–3 hidden layers, 64–256 nodes, ReLU, Adam, early stopping), then tune by grid search or random search with CV.

10.7 Regularisation of Neural Networks

Neural networks are highly overparameterised and prone to overfitting. Three regularisation techniques are standard:

10.7.1 L2 Regularisation (Weight Decay)

Add a penalty on the sum of squared weights to the loss:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \sum_l \sum_{j,k} \left(W_{jk}^{(l)} \right)^2$$

Equivalently, the weight update becomes $\mathbf{W}^{(l)} \leftarrow (1 - \eta\lambda)\mathbf{W}^{(l)} - \eta\partial\mathcal{L}/\partial\mathbf{W}^{(l)}$: weights are “decayed” toward zero at each step. This is exactly the neural network analogue of Ridge regression and serves the same purpose — shrinking weights reduces overfitting by penalising large, complex models.

10.7.2 Dropout

Dropout is a network-specific regularisation technique that has no direct analogue in classical statistics:

During **training**: at each forward pass, randomly set each hidden unit to zero with probability p_{drop} (typically 0.2–0.5). This is done independently for each mini-batch. Units that are set to zero do not contribute to the forward pass and receive no gradient update in the backward pass.

During **inference** (prediction): all units are active, but their outputs are scaled by $(1 - p_{\text{drop}})$ to keep expected magnitudes consistent.

Why does dropout regularise? It prevents any single hidden unit from becoming excessively important. The network cannot rely on any particular unit being present, so it must learn redundant representations spread across many units. Equivalently, training with dropout is approximately equivalent to training and averaging an exponential number of different network architectures (each dropout mask defines a different sub-network), providing an ensemble-like variance reduction.

10.7.3 Early Stopping

Monitor the **validation loss** throughout training and stop when it stops decreasing:

1. Split training data into train ($\approx 90\%$) and validation ($\approx 10\%$) sets
 2. After each epoch, compute the validation loss
 3. If the validation loss has not improved for p consecutive epochs (patience p , e.g. $p = 10\text{--}20$), stop training
 4. Restore the weights from the epoch with the best validation loss
- Early stopping is simple, highly effective, and requires no additional hyperparameter tuning beyond the patience value.

Early stopping works because overparameterised neural networks initially fit the true signal (validation loss decreases), then begin fitting noise (validation loss increases while training loss continues to decrease). Stopping at the right point prevents the transition into overfitting.

The three regularisers compared

L2 weight decay: penalises large weights globally; analogous to Ridge; works by biasing the model toward simpler functions.

Dropout: randomly disables units during training; prevents co-adaptation of units; effectively trains an ensemble of sub-networks.

Early stopping: prevents overtraining by monitoring validation performance; effectively limits the number of gradient updates, controlling effective model complexity.

In practice, all three are often used simultaneously. Weight decay and dropout are set as hyperparameters; early stopping is applied during all training runs.

10.8 The Bias-Variance Trade-off in Neural Networks

Neural networks are subject to the same bias-variance trade-off as all statistical models:

Network configuration	Bias	Variance	Remedy
Too small / too few layers	High (cannot capture complex patterns)	Low	Add more layers or wider layers
Too large, no regularisation	Low	High (overfits training data)	Add dropout, L2, early stopping
Too large + regularisation	Low–moderate	Low–moderate	Optimal generalisation

The key insight from modern deep learning: **larger networks with strong regularisation often outperform smaller networks without regularisation**, even if the smaller network is already overparameterised. This runs counter to classical intuition and is still an active research area.

Standardisation is mandatory: unlike tree-based methods, neural networks are highly sensitive to the scale of input features. Features on large scales dominate the gradient updates and prevent other features from being learnt. Always standardise all inputs to zero mean and unit variance **before** training, fitting the scaler on training data only.

10.9 Neural Networks vs. Other Methods

The ISLP textbook includes a comparison on the Ames Housing dataset (Chapter 13), where several methods are compared on test MAE:

Method	Relative test performance
OLS	Weakest
Ridge / Lasso	Better than OLS
Single tree	Similar to OLS
Bagging	Good
Random Forest	Very good
Gradient Boosting	Among the best
Neural Network (deep)	Among the best
SVM	Good

Neural networks and gradient boosting tend to be the top performers on tabular data with sufficient observations. The choice between them depends on: interpretability requirements, data size, whether features require engineering, and computational budget.

Theory Questions – Chapter 10

Q1: Why are activation functions necessary in a neural network? What happens if all activation functions are linear?

A: Activation functions are necessary because they introduce non-linearity into the network. Without non-linearity, stacking multiple layers provides no additional modelling power: a composition of linear functions is itself a linear function. Formally, if layer 1 computes $\mathbf{h}^{(1)} = \mathbf{W}^{(1)}\mathbf{x}$ and layer 2 computes $\mathbf{h}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)}$, then $\mathbf{h}^{(2)} = \mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x} = \tilde{\mathbf{W}}\mathbf{x}$, which is just a single linear transformation. Any L -layer network with linear activations is equivalent to a single-layer linear model, regardless of width or depth. Non-linear activations (ReLU, sigmoid, tanh) break this collapse, allowing the network to represent complex, non-linear functions of the inputs. ReLU is preferred in hidden layers because it is computationally simple, avoids the vanishing gradient problem that plagues sigmoid and tanh at saturation, and produces sparse activations.

Q2: Explain the backpropagation algorithm. What problem does it solve, and what mathematical principle does it use?

A: Backpropagation solves the problem of efficiently computing the gradient of the loss $\mathcal{L}$ with respect to all weights in the network — a computation that is necessary for gradient descent but involves millions of parameters in deep networks. The mathematical principle is the **chain rule of calculus**: if $\mathcal{L}$ depends on $\mathbf{z}^{(L)}$ which depends on $\mathbf{W}^{(L)}$, then $\partial\mathcal{L}/\partial\mathbf{W}^{(L)} = (\partial\mathcal{L}/\partial\mathbf{z}^{(L)}) \cdot (\partial\mathbf{z}^{(L)}/\partial\mathbf{W}^{(L)})$. Backpropagation exploits the layered structure of the network to propagate gradients backward from the output layer to the input layer, reusing intermediate computations. A forward pass first computes and stores all activations $\mathbf{h}^{(l)}$ and pre-activations $\mathbf{z}^{(l)}$. The backward pass then computes the gradient at the output layer and propagates it back through the layers using the chain rule, accumulating gradients for each weight matrix. The entire gradient computation costs approximately twice a forward pass — the algorithm is remarkably efficient given the number of parameters involved.

Q3: Explain how dropout regularises a neural network. How does it differ during training versus inference, and why does it work?

A: During training, dropout randomly sets each hidden unit to zero with probability p_{drop} at each forward pass, independently across mini-batches. This means no unit can be relied upon to always be present, so the network cannot develop strong co-adaptations between units (e.g. unit A learning to correct the specific errors of unit B). Instead, each unit must learn a robust representation that is useful

independently. During inference, all units are active, but their outputs are multiplied by $(1 - p_{\text{drop}})$ to keep the expected total input to each subsequent layer the same as during training. Dropout works for three related reasons: (1) it prevents co-adaptation of hidden units, forcing the network to learn more redundant and robust features; (2) it is approximately equivalent to training and averaging an exponential number of sub-networks (one per dropout mask), providing ensemble-like variance reduction; and (3) it acts similarly to L2 regularisation in practice by preventing individual weights from growing too large. Dropout is most effective in the middle hidden layers and is typically not applied to the input or output layers.

11 Chapter 11 – Unsupervised Learning

All methods studied so far are **supervised**: every observation carries a label Y that guides learning. Unsupervised learning removes this supervision — we have only a matrix of features $\mathbf{X}$ and seek to discover structure, patterns, or compact representations within the data. There is no definitive “right answer” to compare against, which makes unsupervised learning simultaneously more exploratory and harder to evaluate. ISLP Chapter 12 covers three pillars: **Principal Component Analysis (PCA)** for dimension reduction, **K -means clustering** for partition-based grouping, and **hierarchical clustering** for tree-structured grouping.

11.1 Principal Component Analysis

11.1.1 Motivation and Intuition

When p is large, visualisation and modelling become difficult. PCA seeks a low-dimensional representation of the data that retains as much variation as possible. The key idea: find a sequence of orthogonal directions in $\mathbb{R}^p$ such that the first direction captures the greatest variance in the data, the second (orthogonal to the first) the next greatest, and so on.

These directions are called **principal components** (PCs). Projecting the data onto the first $M \ll p$ PCs yields an $n \times M$ matrix that compresses the original data while discarding only the least variable — and presumably least informative — directions.

11.1.2 Loading Vectors and Scores

The **first principal component loading vector** $\phi_1 = (\phi_{11}, \phi_{21}, \dots, \phi_{p1})^\top$ solves:

$$\phi_1 = \arg \max_{\phi: \|\phi\|=1} \text{Var} \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right) = \arg \max_{\phi: \|\phi\|=1} \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2$$

The unit-norm constraint $\|\phi_1\| = 1$ prevents the trivial solution of making the loadings arbitrarily large. The **first principal component score** for observation i is the projection:

$$z_{i1} = \sum_{j=1}^p \phi_{j1} x_{ij} = \phi_1^\top \mathbf{x}_i$$

The **second loading vector** ϕ_2 maximises variance subject to the same unit-norm constraint **and** the additional orthogonality constraint $\phi_2 \perp \phi_1$. This continues for all subsequent components. The scores $z_{im} = \phi_m^\top \mathbf{x}_i$ form the columns of the **score matrix** $\mathbf{Z}$ ($n \times M$).

PCA via the singular value decomposition

In practice, PCA is computed via the **singular value decomposition** (SVD) of the (mean-centred) data matrix $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, where $\mathbf{U}$ is $n \times p$ orthogonal, $\mathbf{D}$ is diagonal with singular values $d_1 \geq d_2 \geq \dots \geq d_p \geq 0$, and $\mathbf{V}$ is $p \times p$ orthogonal. The columns of $\mathbf{V}$ are the loading vectors; the columns of $\mathbf{U}\mathbf{D}$ are the score vectors. The singular values relate to variance: the variance explained by the m -th PC is d_m^2/n .

Mean-centring is mandatory. PCA measures variance, and variance is relative to the mean. If the data are not centred, the first PC will primarily capture the overall level of the variables rather than genuine variation around the mean.

11.1.3 Proportion of Variance Explained

The **total variance** of the data is $\sum_{j=1}^p \text{Var}(X_j)$. After mean-centring, this equals $\sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2 = \frac{1}{n} \sum_{m=1}^p d_m^2$.

The **proportion of variance explained (PVE)** by the m -th PC is:

$$\text{PVE}_m = \frac{d_m^2}{\sum_{l=1}^p d_l^2} = \frac{\frac{1}{n} \sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2}$$

The PVEs sum to 1 across all p components. Plotting PVE_m against m gives the **scree plot**.

11.1.4 Choosing the Number of Components

There is no single correct answer; the choice is typically guided by:

- **Scree plot / elbow rule:** plot PVE (or cumulative PVE) vs. m and look for an “elbow” — the point beyond which additional components add little additional variance. This is subjective but widely used.
- **Cumulative PVE threshold:** choose M such that the cumulative PVE exceeds a target, e.g. 80% or 90%.
- **Downstream task performance:** if PCA is used as a preprocessing step (e.g. before KNN or regression), use cross-validation on the downstream task to select M .

Scaling before PCA

If the variables are measured on different scales (e.g. income in thousands of dollars vs. age in years), the variable with the larger variance will automatically dominate the first PC. In this case, **standardise** all variables to zero mean and unit variance before running PCA. If the variables are on the same scale (e.g. gene expression measurements), scaling may not be necessary — but it must be a deliberate choice, not an oversight.

11.1.5 The Biplot

A **biplot** displays both the PC scores (observations, usually shown as points) and the loading vectors (variables, shown as arrows) in the same two-dimensional space of the first two PCs. The direction and length of an arrow represent how strongly the corresponding variable contributes to PC1 and PC2. Observations projected onto an arrow’s direction have high values for that variable. The biplot is a powerful exploratory tool for simultaneously identifying observation clusters and understanding which variables drive each principal component.

1. **Mean-centre** each variable: $\tilde{x}_{ij} = x_{ij} - \bar{x}_j$ (and standardise if scales differ)
2. Compute the SVD of $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$
3. Loading vectors: columns of $\mathbf{V}$ (i.e. $\phi_m = \mathbf{v}_m$, the m -th column)
4. Score matrix: $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}$ (equivalently, $z_{im} = \tilde{\mathbf{x}}_i^\top \phi_m$)
5. PVE of m -th PC: $d_m^2 / \sum_l d_l^2$
6. Choose M via scree plot or cumulative PVE threshold

11.1.6 PCA for Regression: Principal Components Regression

PCR uses the first M PC scores $Z_1, \dots, Z_M$ as predictors in an OLS regression (replacing the original p predictors). Because the PCs are orthogonal, there is no multicollinearity. PCR implicitly assumes the directions of greatest variance in $\mathbf{X}$ are also the most predictive for Y — an assumption that is often but not always justified. PCR is not a feature selection method: each PC is a linear combination of all original predictors.

11.2 K-Means Clustering

11.2.1 Problem Formulation

K -means clustering partitions the n observations into K non-overlapping, non-empty clusters $C_1, C_2, \dots, C_K$. The goal is to minimise the **within-cluster variation (WCV)**:

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \text{WCV}(C_k)$$

The standard measure of within-cluster variation is squared Euclidean distance:

$$\text{WCV}(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2$$

This is equivalent to minimising the **total within-cluster sum of squares**:

$$\sum_{k=1}^K \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2 = \sum_{k=1}^K |C_k| \sum_{j=1}^p \text{Var}_k(X_j)$$

where $\bar{x}_{kj}$ is the mean of the j -th feature in cluster k (the centroid).

11.2.2 The K -Means Algorithm

Exact minimisation of the WCV is NP-hard (requires checking all K^n assignments). The standard algorithm finds a good local minimum:

1. Randomly assign each of the n observations to one of K clusters (random initialisation)
2. **Repeat until convergence:**
 - (a) **Assignment step:** for each observation i , assign it to the cluster k whose centroid $\bar{\mathbf{x}}_k$ is closest:
 $C(i) = \arg \min_k \|\mathbf{x}_i - \bar{\mathbf{x}}_k\|^2$
 - (b) **Update step:** recompute each centroid as the mean of all observations currently assigned to it:
 $\bar{\mathbf{x}}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$
3. Convergence is guaranteed (WCV never increases) and typically occurs within tens of iterations

Because K -means finds a local minimum that depends on the random initialisation, it is standard practice to run the algorithm multiple times with different random starts and keep the solution with the smallest total WCV.

Why convergence is guaranteed

Each step of the algorithm cannot increase the total WCV. In the assignment step, each observation moves to the nearest centroid, reducing or maintaining its individual contribution to WCV. In the update step, the centroid is the unique minimiser of the sum of squared distances to all cluster members (the mean minimises squared error). Since there are finitely many possible assignments and WCV is bounded below by zero, the algorithm must converge.

11.2.3 Choosing the Number of Clusters K

Unlike supervised methods, there is no held-out label to guide model selection. Several heuristics exist:

Elbow method: plot the total within-cluster sum of squares (TWCSS) against K . As K increases, TWCSS always decreases (adding more clusters can only reduce within-cluster variance). Look for an elbow — a value of K beyond which the rate of decrease slows markedly. This is subjective.

Silhouette analysis: for each observation i , compute:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the mean distance from i to all other observations in its cluster (within-cluster cohesion) and $b(i)$ is the mean distance from i to all observations in the nearest other cluster (between-cluster separation). The silhouette score $s(i) \in [-1, 1]$: values close to 1 indicate the observation is well-matched to its cluster, close to 0 means it is on the boundary between clusters, and negative values suggest it may be in the wrong cluster. Choose K to maximise the average silhouette score.

Gap statistic: compares the observed TWCSS to the expected TWCSS under a null reference distribution (data with no cluster structure). A large gap indicates the data has more structure than expected by chance. Choose K where the gap first exceeds the gap for $K + 1$ minus one standard error.

K -means requires standardisation and fixed K

K -means uses Euclidean distance, so variables on larger scales dominate. Always standardise features before clustering. Additionally, K must be specified in advance — this is a significant limitation. Hierarchical clustering avoids this requirement.

11.3 Hierarchical Clustering

11.3.1 Motivation

Hierarchical clustering is an alternative to K -means that does not require specifying K in advance. Instead, it builds a **dendrogram** — a tree-shaped diagram that shows the nested grouping of observations at every possible level of similarity. The analyst can then choose a level of the tree (corresponding to a number of clusters) after inspection.

The most common form is **agglomerative** (bottom-up) hierarchical clustering: start with each observation as its own cluster, then successively merge the two most similar clusters until a single cluster containing all observations remains.

11.3.2 The Agglomerative Algorithm

1. Begin with n clusters, one per observation. Compute the $n \times n$ dissimilarity matrix $\mathbf{D}$ where $d_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$
2. Find the pair of clusters (A, B) with the smallest inter-cluster dissimilarity: $(A, B) = \arg \min_{A \neq B} d(A, B)$
3. Merge A and B into a single new cluster $A \cup B$. Record the merge height as $d(A, B)$
4. Update the dissimilarity matrix: compute the dissimilarity between $A \cup B$ and all remaining clusters using the chosen **linkage** criterion
5. Repeat steps 2–4 until only one cluster remains

The resulting sequence of merges defines the dendrogram. The **height** of a merge on the dendrogram corresponds to the dissimilarity at which the two groups were joined — higher merges indicate groups that are more dissimilar. Cutting the dendrogram horizontally at height h yields the clusters that exist at that dissimilarity level.

11.3.3 Linkage Methods

The linkage criterion determines how the dissimilarity between two clusters (each containing multiple observations) is defined:

Linkage	Definition	Properties
Complete	$d(A, B) = \max_{i \in A, j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$ — maximum pairwise dissimilarity	Tends to produce compact, roughly equal-sized clusters; sensitive to outliers (the worst-case pair dominates)
Single	$d(A, B) = \min_{i \in A, j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$ — minimum pairwise dissimilarity	Can produce long, “chained” clusters (one observation at a time joins a growing chain); very sensitive to bridges between clusters
Average	$d(A, B) = \frac{1}{ A B } \sum_{i \in A} \sum_{j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$ — mean pairwise dissimilarity	Compromise between complete and single; less sensitive to outliers than complete
Ward’s	Minimise the increase in total within-cluster variance upon merging	Tends to produce compact, balanced clusters; similar to K -means objective; often preferred for general use

Complete and average linkage are the most commonly used in practice. **Ward’s linkage** is popular when the goal is to produce clusters similar to K -means. **Single linkage** is rarely preferred because of its chaining tendency.

11.3.4 Dissimilarity Measures

The algorithm requires a dissimilarity measure $d(\mathbf{x}_i, \mathbf{x}_j)$ between observations. The choice matters and should reflect the problem:

- **Euclidean distance:** $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{\sum_k (x_{ik} - x_{jk})^2}$ — standard default; sensitive to scale, so standardise first.
- **Manhattan distance:** $d(\mathbf{x}_i, \mathbf{x}_j) = \sum_k |x_{ik} - x_{jk}|$ — more robust to outliers than Euclidean.
- **Correlation-based distance:** $d(\mathbf{x}_i, \mathbf{x}_j) = 1 - \text{corr}(\mathbf{x}_i, \mathbf{x}_j)$ — appropriate when the overall level (magnitude) of observations is not relevant, only their relative patterns (e.g. gene expression profiles: a gene expressed at level 10, 20, 30 and one at level 100, 200, 300 should be considered similar).

For **categorical variables**, the most common dissimilarity is the proportion of features on which two observations differ.

Reading a dendrogram

Two observations are similar if the height at which they first merge (their "join height") is small. Do not compare the height of leaves directly — two leaves close to each other on the page may join at a high height if they are assigned to different early clusters. The meaningful quantity is the **height of the common ancestor node**, not the horizontal distance between leaves.

To obtain K clusters from a dendrogram: draw a horizontal cut line at an appropriate height and count the number of branches crossing it — this is K . Choosing the cut height is analogous to choosing K in K -means.

11.3.5 Dendrogram and Cluster Stability

One attraction of hierarchical clustering is that the dendrogram displays the clustering at every level simultaneously, allowing the analyst to judge which cut is most natural. A **large gap** between successive merge heights suggests a natural number of clusters (the clusters at the lower level are quite different from each other, and the merge at the gap height is costly). Conversely, if merge heights increase gradually with no distinct gaps, there may be no strong cluster structure in the data.

11.4 Comparing Clustering Methods

Feature	K -Means	Hierarchical (Agglomerative)
Number of clusters	Must specify K	Inferred from dendrogram
Result type	Flat partition	Nested hierarchy (dendrogram)
Algorithm	Iterative (Lloyd's)	Sequential merging
Objective	Minimise WCV (Euclidean)	Minimise linkage-based dissimilarity
Sensitivity	Random initialisation	Linkage and distance choice
Scalability	Scales well to large n	$O(n^2)$ memory; slow for large n
Handles non-spherical clusters?	Poorly (assumes convex clusters)	Depends on linkage (single: yes, complete: no)

11.5 Practical Considerations

Scaling/standardisation: both K -means and PCA are sensitive to variable scales. Standardise to zero mean and unit variance unless variables are already on the same scale or there is a domain reason not to.

Reproducibility: K -means is random (initialisation) and different runs may give different solutions. Always set a random seed and run multiple initialisations. Hierarchical clustering is deterministic given the data, dissimilarity measure, and linkage.

Validating cluster structure: unlike supervised learning, there is no test error. Use domain knowledge, stability analysis (repeat on bootstrap samples and measure if clusters are preserved), or internal validation metrics (silhouette, gap statistic) to assess whether structure is meaningful or artefactual.

The curse of dimensionality in clustering: Euclidean distance becomes less meaningful in high dimensions — all pairwise distances concentrate around the same value. In high- p settings, consider applying PCA first (dimensionality reduction) and then clustering on the leading PCs.

Theory Questions – Chapter 11

Q1: What are principal components? Define the loading vector and the score vector for the first PC, and explain how the second PC differs from the first.

A: Principal components are a sequence of orthogonal directions in the p -dimensional feature space that successively capture the maximum remaining variance in the data. The **first loading vector** $\phi_1 \in \mathbb{R}^p$ (with $\|\phi_1\| = 1$) is the direction along which the projected data has the largest variance. Formally, $\phi_1 = \arg \max_{\|\phi\|=1} \text{Var}(\tilde{\mathbf{X}}\phi)$. The **first score vector** $\mathbf{z}_1 \in \mathbb{R}^n$ contains the projections of all n (mean-centred) observations onto this direction: $z_{i1} = \phi_1^\top \tilde{\mathbf{x}}_i$. These scores are the coordinates of the observations in the new PC space and are used as reduced-dimension representations. The **second PC** is found by solving the same maximisation problem with the additional constraint that $\phi_2 \perp \phi_1$ — i.e. the second loading vector must be orthogonal to the first. This constraint ensures the second PC captures variance not already captured by the first. The scores $z_{i2} = \phi_2^\top \tilde{\mathbf{x}}_i$ are therefore uncorrelated with the first scores. Subsequent PCs follow the same pattern: each is the direction of maximum remaining variance orthogonal to all previous PCs.

Q2: Describe the K -means algorithm step by step. Why is convergence guaranteed? What is the main limitation of K -means, and how is it typically addressed in practice?

A: The K -means algorithm begins by randomly assigning each of the n observations to one of K clusters. It then iterates between two steps: (1) the **assignment step**, where each observation is reassigned to the cluster whose centroid (mean) is nearest in Euclidean distance, and (2) the **update step**, where each centroid is recomputed as the mean of all observations currently in that cluster. These steps repeat until assignments no longer change. Convergence is guaranteed because each step monotonically decreases the total within-cluster sum of squares (TWCSS): in the assignment step, each observation moves to a strictly nearer (or equally near) centroid; in the update step, the cluster mean is the unique minimiser of squared Euclidean distance within the cluster. Since TWCSS is bounded below by zero and there are finitely many distinct assignments, the algorithm must terminate. The main limitation is that K -means finds a **local minimum** of TWCSS whose quality depends on the random initialisation. The same data with different starting assignments can yield very different clusterings. The standard remedy is to run the algorithm many times (e.g. 10–100) with different random initialisations and retain the solution achieving the lowest TWCSS.

Q3: Compare complete linkage and single linkage in hierarchical clustering. What kinds of clusters does each produce, and in what situations might you prefer one over the other?

A: **Complete linkage** defines the dissimilarity between two clusters as the *maximum* pairwise distance between any observation in the first cluster and any observation in the second: $d_{\text{complete}}(A, B) = \max_{i \in A, j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$. Because it uses the worst-case pair, complete linkage tends to produce **compact, roughly spherical clusters of similar size** — two clusters are not merged until even their most distant members are fairly close. It is, however, sensitive to outliers: a single extreme observation can greatly inflate $d_{\text{complete}}(A, B)$ and delay a natural merge. **Single linkage** defines the dissimilarity as the *minimum* pairwise distance: $d_{\text{single}}(A, B) = \min_{i \in A, j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$. Because it merges on the best-case pair, single linkage tends to produce **elongated, chained clusters**: a single close observation can "bridge" two otherwise distant groups, causing them to merge into a long chain. This chaining tendency makes single linkage poorly suited to discovering compact, separated clusters, but it excels at detecting clusters of unusual shapes (e.g. crescent-shaped or filamentary structures). In most practical settings with roughly convex clusters, complete or average linkage (or Ward's) is preferred over single linkage.